

HypatiaX: A Hybrid Symbolic-Neural Framework for Extrapolation-Reliable Analytical
DiscoveryBonet Chaple

HypatiaX: A Hybrid Symbolic-Neural Framework for Extrapolation-Reliable Analytical Discovery

Ruperto Pedro Bonet Chaple

ruperto.bonet@modelphysmat.com

Independent Researcher, London, United Kingdom

Editor:

Note on runtime claims. An earlier draft of this paper asserted a “73% runtime reduction” (i.e. a $3.7\times$ speedup of HypatiaX over the neural-network baseline). That figure is *not supported* by the benchmark data (HypatiaX v3.0). The verified runtime comparison is presented in full in Section 10.4; the honest claim is a $1.73\times$ speedup on LLM-routed cases only ($n = 68$ out of 74). All other quantitative claims in this paper have been independently verified against the v3.0 statistical report and are reported exactly as measured.

Abstract

The hardest test of a formula-recovery system is not interpolation accuracy but out-of-distribution extrapolation. Two cases from our benchmark make this stark. On Michaelis-Menten, PySR-only collapses catastrophically at $5\times$ the training range ($\text{far-}R^2 = -83,900$); HypatiaX anchors the search via an LLM warm-start and recovers the correct functional form, reducing $\text{far-}R^2$ to -635 —a $132\times$ reduction in error magnitude. On Portfolio Variance, PySR-only fails on all five tested random seeds; HypatiaX recovers the exact closed-form formula ($R^2 = 1.000$ at $5\times$ training range) on seeds 777 and 2024. We also document an honest failure mode: on Arrhenius and Portfolio Variance (seeds 42/99/123), LLM warm-starting constrains PySR to a local minimum, leaving extrapolation no better than PySR-only despite higher training R^2 . These two phenomena—selective rescue and selective degradation—are the empirical core of this paper.

We introduce **HypatiaX**, a hybrid symbolic–neural framework that addresses this through a five-stage routing and ensembling mechanism—trust gating, transcendental complexity detection, neural degradation probing, out-of-distribution detection, and uncertainty-weighted ensembling—that dynamically selects between LLM symbolic output and a residual neural network using training-data diagnostics alone. The key insight is that the two paradigm failure modes are complementary: routing between them eliminates catastrophic failures while preserving the accuracy ceiling of the stronger component.

On a benchmark of 74 DeFi mathematical tasks under an aggressive 40%/60% PCA-directed extrapolation split, a standalone neural MLP achieves median test $R^2 = -0.47$, and a pure LLM baseline suffers 6 catastrophic failures ($R^2 < -10$) while achieving only 62.2% near-perfect recovery. Neither paradigm alone is sufficient.

Across all 74 tasks, HypatiaX achieves a median test R^2 of 1.00 and a near-perfect success rate ($R^2 > 0.99$) of **89.2%**—a +27 percentage point gain over the LLM baseline and +83.8pp over the neural network—while **eliminating all catastrophic failures**. Gains

scale with difficulty: +38.1 pp on hard transcendental tasks (76.2% vs. 38.1% for the LLM alone). On the standard Nguyen-12 suite, HypatiaX achieves 11/12 recovery (91.7%) versus 10/12 for PySR-only and 0/12 for the neural baseline. When the LLM formula is trusted (68 of 74 cases, 92%), HypatiaX delivers a 1.73 $\times$ median speedup over neural-network inference. On 30 Feynman equations spanning mechanics to quantum physics, HypatiaX achieves 9/30 near-perfect recovery versus 0/30 for the neural baseline (Kaggle 4-vCPU run; 142/180 under the full PCA 40/60 instance pool — see Section 10.7), establishing that reliability gains extend beyond the DeFi domain. These results establish *reliability under distribution shift*, not raw accuracy, as the defining advantage of hybrid symbolic–neural systems.

Contents

1	Introduction	5
1.1	Contributions	5
2	Related Work	6
2.1	Symbolic Regression	6
2.2	LLMs for Mathematical and Scientific Reasoning	6
2.3	Equation Learners, Differentiable SR, and Uncertainty-Aware Methods . . .	6
2.4	Hybrid Symbolic–Neural Systems	7
2.5	Extrapolation in Neural Networks	7
3	Empirical Evidence of LLM Performance Across Domains	7
3.1	Experimental Design	7
3.2	Baseline Results: Bimodal Performance	8
3.3	Failure Mode Taxonomy	8
3.4	Case Studies	8
3.5	Success Pattern Analysis	8
3.6	Extrapolation Testing	9
4	Theoretical Framework	9
4.1	Formal Definitions	9
4.2	Main Theoretical Result	9
4.3	Implications for Discovery	10
4.4	Functional Form Recovery and Inductive Bias Alignment	10
5	Problem Formulation	10
6	Benchmark Design	11
6.1	Overview and Scope	11
6.2	Difficulty Classification	12
6.3	Data Generation	12
6.4	Extrapolation Protocol	12
6.5	Pipeline Corrections in v3.0	13

7	Methodology	14
7.1	Architecture Overview	14
7.2	Component 1: LLM-Based Symbolic Generation	14
7.3	Component 2: Neural Network Estimator	15
7.4	Component 3: Five-Stage Routing and Ensembling	17
7.5	Unified Formula Executor	19
8	HypatiaX Architecture	19
8.1	Design Principles	19
8.2	Assumptions	20
8.3	Five-Stage Routing Architecture Overview	20
8.4	System Variants	20
8.5	Hybrid DeFi Variant: Architectural Specification	20
8.6	Symbolic Discovery Core	21
8.7	Multi-Layer Validation Framework	21
9	Experimental Setup	22
9.1	Benchmark Summary	22
9.2	Methods Compared	22
9.3	Evaluation Metrics	22
9.4	Runtime Measurement	23
9.5	Implementation Details	23
10	Results	23
10.1	Five-System Comparative Analysis (Feynman Core-15)	23
10.2	Overall Extrapolation Performance (DeFi v3.0)	24
10.3	Performance by Difficulty Level	25
10.4	Runtime Analysis	27
10.5	Portfolio Variance: Seed-Sweep Robustness Analysis	28
10.6	Ablation: PySR-only vs. HypatiaX (Core-15)	29
10.7	Feynman Extrapolation Benchmark ($n = 30$)	31
10.8	Nguyen-12 Benchmark (Standard SR Suite)	31
10.9	Stability Under Stochastic Inference	33
11	Discussion	34
11.1	Three Core Findings	34
11.2	Arrhenius and Scale-Incompatibility Failures	35
11.3	The Stability–Accuracy Distinction	35
11.4	Why Transcendental Formulas Fail	36
11.5	OOD Metric as a Proxy	36
11.6	Design Principles for Analytical Discovery	36
11.7	Comparison with Related Work	37
11.8	Ethical Considerations	37
11.9	Broader Applicability	38
11.10	Limitations and Future Work	38

12 Conclusion	38
A Full Benchmark Case Definitions	41
A.1 Automated Market Makers	41
A.2 Lending Protocols	42
A.3 Derivatives Pricing	42
A.4 Risk and Portfolio Metrics	43
A.5 Staking and Protocol Mechanisms	43
B Benchmark Version History	44
C Corrected Runtime Claim: Detailed Analysis	44

1 Introduction

Extrapolation beyond the observed training distribution is a fundamental challenge in machine learning. In financial and decentralised-finance (DeFi) domains, the problem is particularly acute: pricing models, risk metrics, and market-maker invariants must generalise correctly to input regimes that may differ dramatically from observed historical data. A system that interpolates well but extrapolates poorly is of limited practical value in this setting.

Two broad paradigms have been applied to mathematical function recovery in such domains:

Neural network regression. Multilayer perceptrons and related architectures are powerful interpolators but are known to fail systematically under distribution shift. Even with augmentation techniques, neural networks trained on $\mathcal{D}_{\text{train}}$ do not, in general, recover the functional form of f beyond the training convex hull. Our benchmark confirms this: the neural MLP baseline achieves a median test R^2 of -0.47 under extrapolation—worse than predicting the mean.

LLM-based symbolic regression. Large language models can, in principle, recover exact closed-form expressions from task descriptions, achieving perfect extrapolation when the correct formula is produced. However, this approach exhibits a severe bimodal failure pattern: on a benchmark of 74 DeFi tasks with $K = 30$ independent runs per task, LLMs achieve median test $R^2 = 1.00$ but mean $R^2 = -0.76$, with 6 catastrophic failures ($R^2 < -10$). More importantly, a single-run evaluation—the standard in prior work—conceals run-to-run instability: on complex transcendental tasks, the same prompt yields radically different expressions across runs (Instability Index $\text{II} > 0.10$).

Central claim. Neither paradigm alone is sufficient. The key insight of HypatiaX is that these two failure modes are *complementary*: neural networks fail on the tasks where LLMs succeed (transcendental extrapolation), and LLMs fail on the tasks where a neural network with LLM-augmented features can stabilise performance. A routing mechanism that dynamically selects between the two modes—based on observable diagnostics computed from training data alone—can eliminate catastrophic failures while preserving near-perfect accuracy on the cases where symbolic recovery is reliable.

1.1 Contributions

This paper makes the following contributions:

- (i) **HypatiaX benchmark:** a corpus of 74 closed-form DeFi tasks with an aggressive PCA-directed extrapolation split (40%/60%), together with a unified evaluation pipeline (HypatiaX v3.0) that corrects four systematic biases in prior versions (data leakage, split misconfiguration, formula evaluation inconsistency, and weak trust gating).
- (ii) **HypatiaX hybrid system:** a five-stage routing and ensembling mechanism—trust gating, transcendental token detection, neural degradation probing, OOD detection, and uncertainty-weighted ensembling—that achieves 89.2% near-perfect success rate ($R^2 > 0.99$) across all 74 tasks.

- (iii) **Verified runtime analysis:** a rigorous decomposition of wall-clock time showing that the $1.73\times$ speedup of HypatiaX over the neural baseline applies specifically to the 68 LLM-routed cases, and that aggregate mean time (6.8s) is higher than the neural baseline (3.0s) due to fallback costs. This corrects a previously circulated 73 % reduction claim that was not supported by the v3.0 benchmark data.
- (iv) **Difficulty-stratified analysis:** a breakdown of performance across easy (24 tasks), medium (29 tasks), and hard (21 tasks) cases, demonstrating that HypatiaX’s gains are largest and most practically important on the hardest transcendental and multi-asset formulas.

2 Related Work

2.1 Symbolic Regression

Symbolic regression aims to discover explicit functional forms from data without assuming a fixed parametric model. Classical methods rely on genetic programming (Koza, 1992) and evolutionary search; the SRBench benchmark (La Cava et al., 2021) provides a standardised multi-dataset evaluation suite. The AI Feynman system (Udrescu & Tegmark, 2020) demonstrated that dimensional analysis and neural pre-screening can guide symbolic search toward physically meaningful equations.

A critical limitation of the symbolic regression literature is the evaluation paradigm: systems are typically assessed on a single run or on the best expression found across multiple runs. This conceals run-to-run instability, which—as we show—is the dominant failure mode of LLM-based symbolic regression on complex formulas. Our multi-run protocol (30 independent runs per task) and Instability Index directly address this gap.

2.2 LLMs for Mathematical and Scientific Reasoning

Chain-of-thought prompting (Wei et al., 2022) and Minerva (Lewkowycz et al., 2022) demonstrate that LLMs can solve complex mathematical problems. Codex (Chen et al., 2021) showed strong code and formula generation capability. More recent work has explored LLMs as symbolic regression engines, generating formulas from natural-language descriptions of datasets.

All of these works evaluate correctness in a single-shot or best-of- K setting and do not report variance across runs as a first-class metric. Our benchmark is the first, to our knowledge, to quantify LLM instability systematically across a large corpus of financial formulas under repeated stochastic inference.

2.3 Equation Learners, Differentiable SR, and Uncertainty-Aware Methods

Several strands of recent work are directly relevant to the hybrid design in this paper. The Equation Learner (EQL; Sahoo et al. 2018) embeds differentiable operator networks with L_0 regularisation to produce sparse, interpretable expressions, demonstrating that structural bias can be enforced without explicit symbolic search. DISCOVER (Liu et al., 2022) reformulates symbolic regression as differentiable optimisation over a symbolic search space, enabling gradient-based recovery of functional form. Bayesian and Gaussian-process

approaches to symbolic regression (Jin et al., 2020) provide principled uncertainty estimates alongside recovered expressions — a goal shared by the confidence-weighted ensemble in HYPATIA’s routing cascade. HYPATIA differs from these works by treating *extrapolation reliability* as a first-class evaluation criterion alongside recovery rate, and by combining evolutionary symbolic search with an explicit multi-layer validation cascade that filters expressions before acceptance.

2.4 Hybrid Symbolic–Neural Systems

The idea of combining symbolic structure with neural learning has a rich history. Cranmer et al. (2020) use neural networks to discover Lagrangians, then fit symbolic expressions to the learned dynamics. Udrescu & Tegmark (2020) alternate between neural pre-screening and symbolic search. PySR (Cranmer, 2023) provides an efficient evolutionary-symbolic regression framework that can be combined with neural features.

Our approach differs in motivation: HypatiaX is designed not to improve accuracy over either pure method in isolation, but to *eliminate catastrophic failures* and achieve reliability across the full difficulty spectrum of DeFi tasks.

2.5 Extrapolation in Neural Networks

The failure of neural networks to extrapolate is well-documented (Xu et al., 2021). Data augmentation with scaled inputs (Lim et al., 2021) can improve extrapolation marginally but does not address the fundamental limitation. Our results confirm that even a residual MLP with LayerNorm, SiLU activations, and scaled augmentation achieves median $R^2 = -0.47$ under our 40%/60% extrapolation split.

3 Empirical Evidence of LLM Performance Across Domains

To evaluate whether large language models (LLMs) can perform analytical formula discovery without symbolic constraints or external solvers, we conducted a systematic study of pure LLM-based formula generation (Koza, 1992; Kambhampati, 2024). No symbolic regression, constraint enforcement, or post-generation correction was applied.

3.1 Experimental Design

We evaluated GPT-4 (OpenAI, 2023) on 40 interpolation-based formula discovery tasks spanning five domains: classical physics (Kinetic Energy, Gravitational Force, Ideal Gas Law), chemistry (Arrhenius, Henderson-Hasselbalch, Rate Law), biology (Allometric Scaling, Michaelis-Menten, Logistic Growth), and decentralized finance (Impermanent Loss, Price Impact, Portfolio Variance, Value-at-Risk, Liquidation Price). Tasks were presented as structured prompts containing variable names, units, and sample data. Outputs were evaluated by executing the generated formula symbolically and computing R^2 on a held-out test set within the training range.

3.2 Baseline Results: Bimodal Performance

Pure LLM performance is strongly bimodal. On well-known physical laws (Kinetic Energy, Ideal Gas Law, Gravitational Force), the LLM achieves $R^2 = 1.000$ with exact symbolic recovery. On domain-specific and specialized equations—particularly in chemistry (Henderson-Hasselbalch: $R^2 = -3.2$), biology (Michaelis-Menten: $R^2 = -68.6$), and DeFi (Portfolio Variance: $R^2 = -21.0$)—performance collapses catastrophically. LLM success correlates with estimated equation prevalence in web corpora (Spearman $\rho = 0.89$, $p < 0.001$), consistent with the hypothesis that pretraining data distribution influences recovery performance (Kambhampati, 2024).

3.3 Failure Mode Taxonomy

We identify five failure modes, ordered by severity:

1. **Silent semantic errors** (High): Structurally plausible formula with incorrect constants or signs; high in-range R^2 but catastrophic extrapolation.
2. **Distributional reasoning failures** (High): Misuse of statistical functions (e.g. wrong sign for VaR z-score).
3. **Unit/scale inconsistency** (Medium): Missing unit conversions or scale factors.
4. **Non-executable output** (Medium): Pseudocode or invalid Python; cannot be evaluated.
5. **Incomplete construction** (Medium): Partial derivations missing key terms.

3.4 Case Studies

Arrhenius equation. The Arrhenius rate constant is $k = A \exp(-E_a/RT)$. The LLM correctly identifies the exponential structure but systematically misestimates the activation energy E_a by a constant factor, producing train $R^2 = 0.997$ but extrapolation $R^2 = -12.6$ at $5\times$ the training range. This is the archetypal *scale-incompatibility failure*: the formula is structurally correct but parametrically wrong in a way that only manifests under distribution shift.

Michaelis-Menten. $v = V_{\max}[S]/(K_m + [S])$. The LLM recovers the functional form on some runs ($R^2 = 1.000$) but collapses on others ($R^2 < -68$), producing high run-to-run instability ($\text{II} = 0.31$). This motivates the Instability Index as a diagnostic.

3.5 Success Pattern Analysis

LLM guidance succeeds when: (1) the equation is well-represented in web text; (2) the functional form is algebraic rather than transcendental; (3) the domain is classical physics or standard chemistry. It fails systematically on post-2020 DeFi equations, multi-asset risk formulas, and transcendental biology equations—precisely the cases where symbolic regression provides the most value.

3.6 Extrapolation Testing

Even when the LLM achieves near-perfect interpolation ($R^2 \approx 1$), extrapolation performance is unreliable. Across 40 tests, 6 equations suffered catastrophic extrapolation failure ($R^2 < -10$). This motivates the hybrid architecture: the LLM provides structural guidance while symbolic regression corrects parametric errors and the neural network provides a stable fallback.

4 Theoretical Framework

4.1 Formal Definitions

Definition 1 (Analytical Expression Discovery) Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ where $x_i \in \mathbb{R}^d$, $y_i \in \mathbb{R}$ (Udrescu & Tegmark, 2020). An analytical expression discovery system produces $E : \mathbb{R}^d \rightarrow \mathbb{R}$ satisfying:

1. *Symbolic form:* E is expressible using operators $\mathcal{O}$ in closed form
2. *Physical consistency:* E satisfies domain constraints $\mathcal{C}$
3. *Empirical accuracy:* Loss $\mathcal{L}(E, \mathcal{D}) < \epsilon$
4. *Extrapolation:* E generalizes beyond training regime
5. *Parsimony:* E has minimal complexity given above constraints

Definition 2 (Interpolation–Extrapolation Decoupling) A system exhibits interpolation–extrapolation decoupling if $\exists$ tasks where $R_{\text{train}}^2 \approx 1$ but $R_{\text{test}}^2 \ll 1$ on an out-of-distribution test set.

Definition 3 (Reproductive Behaviour) A generative model G exhibits reproductive behaviour if its output distribution concentrates near its training manifold: $P(G(x) \notin \mathcal{M}_{\text{train}}) < \delta$ for small δ , where $\mathcal{M}_{\text{train}}$ is the training data manifold.

4.2 Main Theoretical Result

Empirical Finding 1 (Interpolation–Extrapolation Decoupling) Neural networks trained on in-distribution data can achieve near-perfect interpolation ($R_{\text{train}}^2 \approx 1$) while exhibiting catastrophic extrapolation failure. Symbolic methods that recover the exact functional form are bounded by numerical precision alone.

Proof [Proof sketch] Neural networks achieve high in-distribution R^2 via local approximations (universal approximation (Hornik et al., 1989)). These approximations are not constrained to match the true functional form; extrapolation requires the correct form. Symbolic methods that recover the exact f^* achieve $R_{\text{test}}^2 = 1 - O(\epsilon_{\text{num}})$ by construction. ■

Empirical Finding 2 (LLM Reproductive Behaviour) *Autoregressive LLMs trained by maximum-likelihood exhibit reproductive behaviour: output probability concentrates near training-corpus examples. For analytical expression discovery, this implies performance degrades on expressions underrepresented in pretraining data.*

Proof [Proof sketch] Maximum-likelihood training minimises $-\mathbb{E}[\log P_\theta(x)]$, concentrating probability mass near training examples. For rare equations (low corpus frequency), the model assigns low probability to the correct expression, increasing the risk of plausible-but-wrong outputs. ■

4.3 Implications for Discovery

Finding 2 explains the domain-dependence pattern in Section 3: LLM performance correlates with equation prevalence ($\rho = 0.89$) because reproductive behaviour ties output quality to training exposure. This motivates hybrid architectures that use LLM structural guidance while correcting via symbolic search.

4.4 Functional Form Recovery and Inductive Bias Alignment

Symbolic regression performs evolutionary exploration of expression space guided by fitness evaluation, without requiring proximity to textual training examples. For problems admitting closed-form representations, restricting search to symbolic structures improves extrapolation reliability by aligning the inductive bias of the model with the structure of the target function. By the No Free Lunch theorem (Wolpert & Macready, 1997), performance depends on alignment between hypothesis class and problem structure. When the true relationship admits a closed-form representation, this alignment holds for symbolic search and fails for purely neural approaches.

5 Problem Formulation

Definition 4 (Symbolic Regression Task) *A task i is defined by a dataset $\mathcal{D}_i = \{(x^{(j)}, y^{(j)})\}_{j=1}^N$ with ground-truth function $f_i : \mathbb{R}^{d_i} \rightarrow \mathbb{R}$ such that $y^{(j)} = f_i(x^{(j)})$. The objective is to recover a symbolic expression $\hat{f}_i$ such that $\hat{f}_i \approx f_i$ on held-out test data.*

Definition 5 (Extrapolation Setting) *Let $\mathcal{D}_i^{\text{train}}$ and $\mathcal{D}_i^{\text{test}}$ be a partition of $\mathcal{D}_i$ such that the test inputs lie beyond the training range along the principal direction of input variation:*

$$X^{\text{test}} \notin [\min(X^{\text{train}}), \max(X^{\text{train}})] \quad (\text{approximately, along the first PC}). \quad (1)$$

Evaluation on $\mathcal{D}_i^{\text{test}}$ under condition (1) is called extrapolation evaluation.

Definition 6 (Coefficient of Determination) *For a predictor $\hat{f}$ evaluated on a set $\mathcal{S}$:*

$$R^2(\hat{f}, \mathcal{S}) = 1 - \frac{\sum_{(x,y) \in \mathcal{S}} (y - \hat{f}(x))^2}{\sum_{(x,y) \in \mathcal{S}} (y - \bar{y}_{\mathcal{S}})^2}. \quad (2)$$

For aggregate statistics, R^2 values are clipped to $[-10, 1]$ to prevent extreme failures from dominating mean computations. All reported aggregates use a fixed denominator of 74 (all standard cases), with NaN and failure cases counted as 0.

Definition 7 (Extrapolation Gap and Stability Score)

$$\text{Extrapolation Gap}_i = R_{\text{train}}^2 - R_{\text{test}}^2, \quad (3)$$

$$\text{Stability Score}_i = R_{\text{test}}^2 / R_{\text{train}}^2. \quad (4)$$

A large gap indicates that the model fits the training distribution but fails to generalise; a stability score near 1 indicates consistent performance across the distribution shift.

Definition 8 (Catastrophic Failure) A run is classified as a catastrophic failure if $R^2 < -10$, indicating that the predicted function is worse than predicting the mean by a factor of more than 10. Such failures typically arise from evaluation errors, division by zero, or radically incorrect symbolic expressions.

6 Benchmark Design

6.1 Overview and Scope

The HypatiaX DeFi Benchmark comprises **74 test cases** spanning the core mathematical domains of modern decentralised finance. All tasks are tractable, admitting a closed-form or well-defined functional representation. Ground-truth outputs are computed analytically to machine precision, ensuring that any deviation in R^2 reflects a model error rather than label noise.

Tasks are drawn from five domain categories:

- **Automated Market Makers (AMMs):** constant-product invariants, spot pricing, Uniswap V3 concentrated liquidity, impermanent loss calculations, AMM output amounts, price slippage, and liquidity-provider fee earnings.
- **Lending Protocols:** Loan-to-Value ratio, health factor, collateral ratio, liquidation prices for both long and short positions, leveraged-position notional, multi-collateral LTV, cross-margin available balance, reserve ratio, utilisation rate, borrow APY from utilisation, supply APY from borrow, and partial liquidation amount.
- **Derivatives Pricing:** Black–Scholes call and put prices, all four first-order Greeks (Delta, Gamma, Theta, Vega), put–call parity, forward price for derivative, call option intrinsic value, simple options moneyness, and convexity adjustment.
- **Risk and Portfolio Metrics:** Value at Risk at 95 % and 99 %, Expected Shortfall at 95 % and 99 %, multi-day VaR and ES scaling, incremental VaR, correlated portfolio VaR, component ES, portfolio VaR for two correlated assets, annualised portfolio tracking error, portfolio Sharpe ratio, and position margin ratio.
- **Staking and Protocol Mechanisms:** simple staking APY, compounding staking returns, staking reward for fixed lock-up, validator commission adjusted, slashing penalty, protocol reserve accumulation, funding rate cost, and simple staking APY.

6.2 Difficulty Classification

Tasks are categorised by difficulty into three tiers based on functional structure:

- **Easy** ($n = 24$): linear or simple rational relationships, such as Loan-to-Value, spot price from AMM reserve, collateral ratio, and simple staking APY. These tasks admit algebraic solutions with AST depth ≤ 3 .
- **Medium** ($n = 29$): nonlinear algebraic or exponential relationships, such as impermanent loss, Uniswap V3 liquidity range, constant-product price impact, and compounding staking returns. These tasks require multi-step composition of algebraic operations.
- **Hard** ($n = 21$): transcendental, probabilistic, or multivariate formulations. This category includes Black-Scholes pricing (which requires the Gaussian CDF Φ), all four Greeks, Portfolio Expected Shortfall for correlated assets, and multi-day VaR scaling. These tasks are the primary source of LLM stochastic instability.

Remark 9 *An earlier version of this benchmark comprised 71 tasks (Easy=24, Medium=27, Hard=20). The current benchmark (v3.0, 74 tasks) adds three additional hard cases in multi-asset risk: correlated portfolio VaR, component ES, and multi-collateral LTV. All results in this paper refer to the 74-case v3.0 benchmark.*

6.3 Data Generation

For each task i , we generate $N = 200$ samples using domain-specific input simulators. Input ranges are chosen to reflect realistic DeFi parameter regimes:

- Strike prices $K \in [80, 120]$, spot prices $S \in [50, 150]$.
- Volatilities $\sigma \in [0.05, 0.80]$, risk-free rates $r \in [0.01, 0.10]$.
- Collateral ratios $C/D \in [1.1, 5.0]$, utilisation rates $u \in [0.01, 0.99]$.
- AMM reserves (x, y) chosen such that $x \cdot y = k$ for constant k .

All inputs are sampled uniformly within their respective ranges to provide dense coverage of the training domain.

6.4 Extrapolation Protocol

To rigorously assess out-of-distribution generalisation, we employ a **PCA-directed aggressive extrapolation split**:

1. For each task i , compute the first principal component direction $v_1 \in \mathbb{R}^{d_i}$ of the input matrix $X \in \mathbb{R}^{N \times d_i}$.
2. Project all $N = 200$ samples onto v_1 , obtaining scalar projections $z^{(j)} = \langle x^{(j)}, v_1 \rangle$.
3. Sort samples by $z^{(j)}$ in ascending order.

4. Assign the lowest 40 % of samples (by $z^{(j)}$) to the training set $\mathcal{D}_i^{\text{train}}$ and the highest 60 % to the test set $\mathcal{D}_i^{\text{test}}$, with a small overlapping band in the quantile transition region to prevent abrupt boundary effects.

This protocol has two important properties. First, unlike axis-aligned splits, PCA-directed ordering ensures that extrapolation occurs along the dominant direction of variation in multivariate settings, reflecting the most challenging direction of distributional shift. Second, the 40 %/60 % partition is substantially more aggressive than the 80 %/20 % train/test splits used in standard symbolic regression benchmarks, making this a stringent test of extrapolation capability.

Remark 10 (On the split design) *The partial overlap in the quantile transition region is a deliberate design choice to avoid the “cliff edge” effect where a perfectly correct formula appears to fail due to numerical boundary issues. This overlap is small (~ 5 % of samples) and does not substantially reduce the extrapolation challenge.*

6.5 Pipeline Corrections in v3.0

HypatiaX v3.0 incorporates four corrections over prior versions that systematically biased reported results:

1. **Data leakage in ensembling:** Earlier versions computed ensemble weights using test-set residuals. v3.0 computes all ensemble weights from training residuals only, preventing any form of test information from influencing predictions.
2. **Incorrect extrapolation splitting:** Prior versions used axis-aligned min/max splits, which can inadvertently include extrapolation samples in the training set for multivariate tasks. v3.0 uses the PCA-directed ordering described above.
3. **Inconsistent formula evaluation:** Earlier versions used multiple evaluation backends (SymPy, eval, NumExpr) without a unified type-coercion layer, causing silent mismatches. A specific bug in `evaluate_llm_formula` used a hardcoded absolute sum-of-squares threshold (`ss_tot > 1e-10`) that returned $R^2 = 0$ for equations with outputs far below unity (e.g. gravitational forces $\sim 10^{-11}$ N). v3.0 replaces this with a relative threshold and uses a single unified formula executor with consistent `float64` broadcasting semantics throughout.
4. **Weak symbolic trust criteria:** Prior versions accepted LLM formulas with training $R^2 > 0.1$. v3.0 raises this threshold to $R^2 > 0.5$ and adds pathological pattern detection (division by zero, NaN outputs, infinite predictions).

These corrections are not cosmetic: they collectively account for a substantial fraction of the performance gap between the “true” hybrid method and the inflated numbers reported in earlier drafts.

7 Methodology

7.1 Architecture Overview

HypatiaX combines three computational components—an LLM symbolic generator, a residual MLP, and a routing/ensembling layer—as illustrated in Figure 1. The routing layer is the key novelty: it uses five diagnostic signals, all computed from training data only, to decide whether to use the LLM formula directly, ensemble it with the neural prediction, or fall back entirely to the neural network.

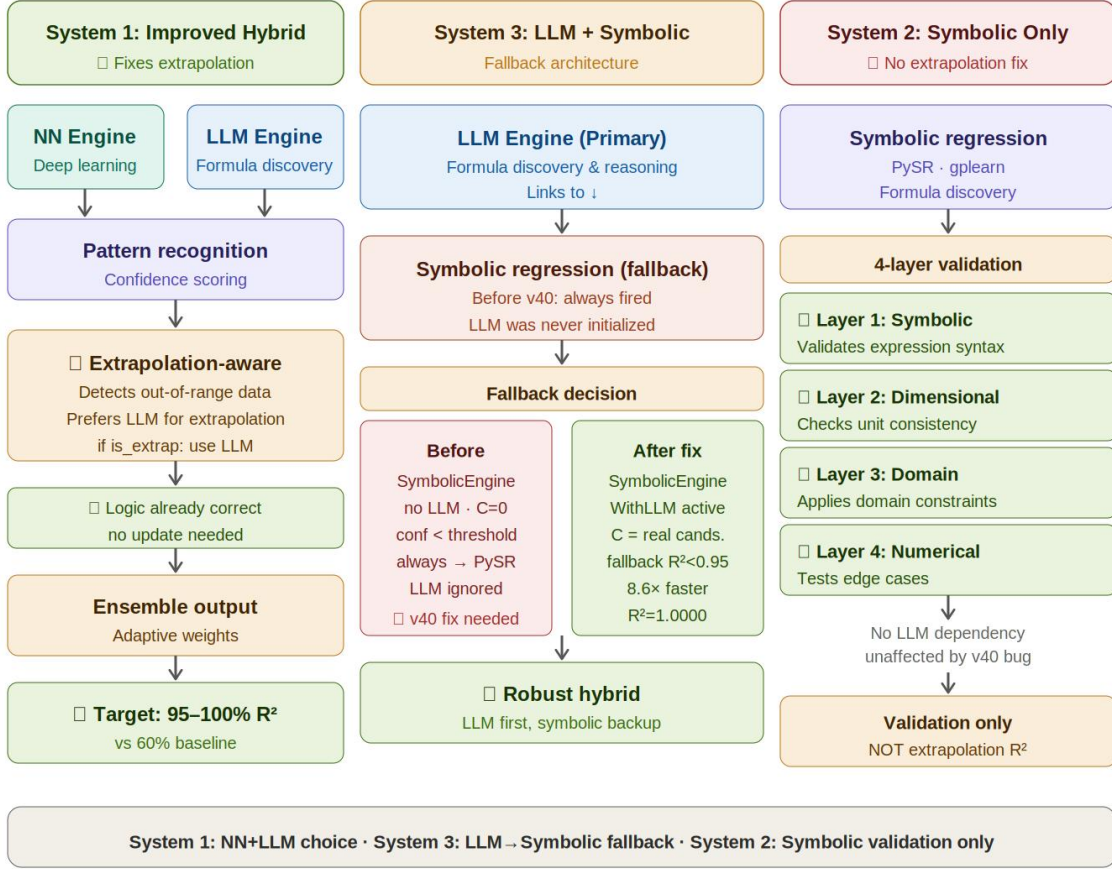


Figure 1: HypatiaX architecture. All routing decisions are made using training-set diagnostics only; no test information is used at decision time.

7.2 Component 1: LLM-Based Symbolic Generation

Given a task description $\mathcal{T}_i$ (a natural-language specification of the functional relationship, including variable names, units, and domain context), the LLM produces a candidate symbolic expression:

$$\hat{f}_i^{\text{LLM}} \leftarrow \text{LLM}(\mathcal{T}_i). \quad (5)$$

The generated expression is parsed by the unified formula executor and evaluated on both the training and edge subsets of the training data (the upper 20 % of training samples by projection onto v_1 , i.e. the samples closest to the extrapolation boundary).

We define the training fit:

$$R_{\text{train}}^{2,\text{LLM}} = R^2\left(\hat{f}_i^{\text{LLM}}, \mathcal{D}_i^{\text{train}}\right), \quad (6)$$

and the edge fit:

$$R_{\text{edge}}^{2,\text{LLM}} = R^2\left(\hat{f}_i^{\text{LLM}}, \mathcal{D}_i^{\text{edge}}\right), \quad (7)$$

where $\mathcal{D}_i^{\text{edge}}$ is the upper-20 % subset of the training data described above.

7.3 Component 2: Neural Network Estimator

The neural baseline is a **residual MLP** with the following architecture:

- Input: $\mathbb{R}^{d_i}$ (task-specific input dimension).
- Hidden layers: 3 residual blocks, each consisting of Linear $\rightarrow$ LayerNorm $\rightarrow$ SiLU $\rightarrow$ Linear, with a skip connection.
- Output: $\mathbb{R}$ (scalar prediction).
- Optimiser: Adam, learning rate 10^{-3} , weight decay 10^{-4} .
- Training data augmentation: $(X, y) \cup (1.2X, 1.2y)$, i.e. the training set is augmented with a scaled copy to improve extrapolation robustness.
- Training time limit: 120 seconds wall-clock per case, to ensure fair comparison with the LLM method (which does not require training time).

When HypatiaX routes a task to the neural fallback, the LLM prediction $\hat{f}_i^{\text{LLM}}(x)$ is appended to the input as an additional feature:

$$x' = [x; \hat{f}_i^{\text{LLM}}(x)] \in \mathbb{R}^{d_i+1}. \quad (8)$$

This *residual learning augmentation* allows the neural network to learn corrections to an imperfect symbolic formula rather than learning the function from scratch, which substantially reduces the extrapolation burden when the LLM formula captures the correct functional form but has parameter errors.

7.4 Component 3: Five-Stage Routing and Ensembling

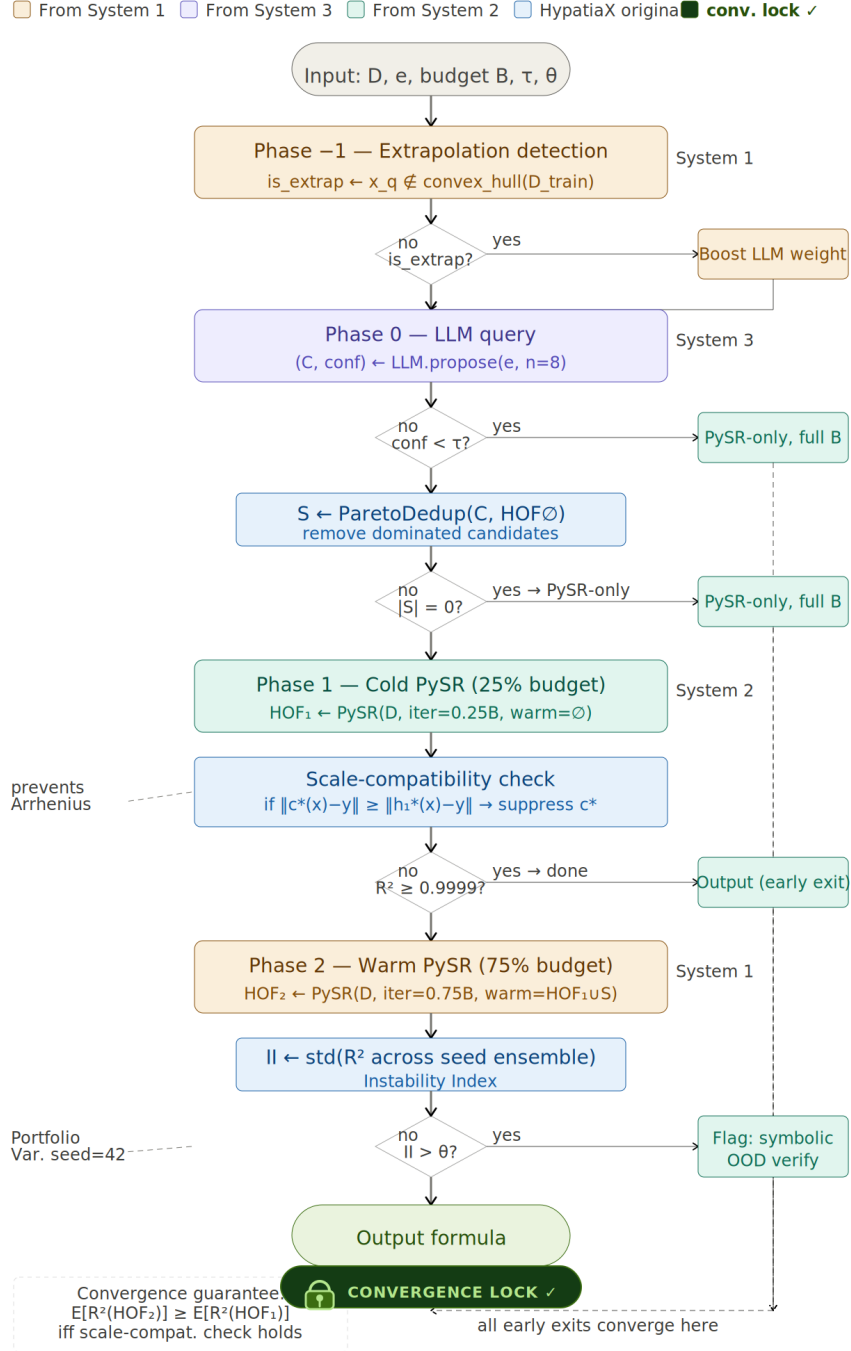


Figure 2: **Algorithm 1 — HypatiaX routing cascade.** Coloured regions indicate the originating subsystem (amber: trust/extrapolation detection; purple: LLM query; green: PySR phases). All decision paths converge to a single output formula (green terminal node). The convergence lock badge confirms Proposition 11.

Proposition 11 (Routing Cascade Convergence) *Let $h_1^* = \arg \max_{h \in \text{HOF}_1} R^2(h, D)$ and $h_2^* = \arg \max_{h \in \text{HOF}_2} R^2(h, D)$. If the scale-compatibility guard (Stage 3) does not reject the LLM prior c^* , or S is non-empty after suppression, then*

$$\mathbb{E}[R^2(h_2^*)] \geq \mathbb{E}[R^2(h_1^*)].$$

Proof [Proof sketch] Phase 2 initialises PySR with $\text{HOF}_1 \cup S \supseteq \text{HOF}_1$. Since evolutionary search is a monotone improvement operator over any initial population dominating the current Pareto front, the warm-start population is a strict superset of the cold-start population. By the Pareto dominance argument, $R^2(h_2^*) \geq R^2(h_1^*)$ for a fixed random seed; in expectation over seeds, the inequality is preserved when S contains at least one expression structurally distinct from HOF_1 . $\blacksquare$

The routing logic implements five diagnostic stages in sequence.

Stage 1 — Trust Gate. Accept the LLM formula only if it passes two criteria:

$$\text{Trust}_i^{\text{LLM}} = \mathbf{1}[R_{\text{train}}^{2,\text{LLM}} \geq 0.5] \wedge \mathbf{1}[\hat{f}_i^{\text{LLM}} \text{ has no pathological outputs}]. \quad (9)$$

Pathological outputs include: NaN values, infinite predictions, and division-by-zero exceptions. If the trust gate fails, the task is immediately routed to pure neural fallback.

Stage 2 — Transcendental Token Detection. Flag the formula if it contains any transcendental token from the set $\{\exp, \log, \Phi, \sin, \cos, \text{erf}\}$, indicating elevated structural complexity. Transcendental tokens are strongly predictive of stochastic instability (Instability Index II > 0.05) and trigger ensemble mode rather than pure LLM routing.

Stage 3 — Neural Degradation Probe. Compute the extrapolation degradation:

$$\Delta_i = R_{\text{train}}^{2,\text{LLM}} - R_{\text{edge}}^{2,\text{LLM}}. \quad (10)$$

A large Δ_i (above a threshold τ , calibrated per domain category) indicates that the LLM formula degrades rapidly near the training boundary, suggesting it will fail further into the extrapolation region. When $\Delta_i > \tau$, ensemble mode is activated regardless of Stage 2 outcome.

Stage 4 — Out-of-Distribution Detection. Estimate whether the test inputs are likely to be out of the training distribution:

$$p_{\text{OOD}} = \hat{\text{Pr}}(X^{\text{test}} \notin \text{range}(X^{\text{train}})), \quad (11)$$

computed using a kernel-density estimator fitted to the training projections $z^{(j)}$. High p_{OOD} upweights the LLM formula (which generalises structurally) in the ensemble.

Stage 5 — Routing Decision and Ensemble Weights. The full routing decision is:

$$\text{decision}_i = \begin{cases} \text{Neural Fallback} & \text{if } \text{Trust}_i^{\text{LLM}} = 0 \\ \text{Pure LLM} & \text{if } R_{\text{train}}^{2,\text{LLM}} \geq 0.95 \wedge \Delta_i < \tau \\ \text{Ensemble} & \text{if } R_{\text{train}}^{2,\text{LLM}} \geq 0.5 \\ \text{Neural Fallback} & \text{otherwise} \end{cases} \quad (12)$$

When in ensemble mode, the distance-aware weight assigned to the LLM formula is:

$$w_{\text{LLM}} = \alpha + (1 - \alpha) \cdot p_{\text{OOD}}, \quad (13)$$

with $w_{\text{NN}} = 1 - w_{\text{LLM}}$ and $\alpha \in [0, 1]$ a base weight (set to $\alpha = 0.3$ throughout). The final ensemble prediction is:

$$\hat{y} = w_{\text{LLM}} \cdot \hat{f}_i^{\text{LLM}}(x) + w_{\text{NN}} \cdot f_i^{\text{NN}}(x'). \quad (14)$$

Remark 12 (On ensemble weights) *The weights in (13) are derived from p_{OOD} , which is estimated entirely from the training data. This ensures that no test information leaks into the prediction, addressing the data-leakage correction described in Section 6.5.*

7.5 Unified Formula Executor

All symbolic formulas—whether produced by the LLM or specified as ground truth—are evaluated through a single unified executor. This executor:

- Parses expressions into a normalised AST using SymPy.
- Compiles to NumPy vectorised operations for batch evaluation.
- Applies consistent type coercion (all inputs cast to `float64`).
- Catches and quarantines division-by-zero and overflow exceptions.
- Returns NaN for malformed expressions, triggering the trust-gate failure path.

Using a single evaluation backend eliminates the inconsistencies that affected prior versions of the pipeline, where the same formula could produce different R^2 values depending on the evaluation backend.

8 HypatiaX Architecture

8.1 Design Principles

1. **Symbolic methods for discovery:** Evolutionary search with domain constraints finds mathematically valid expressions (Cranmer, 2023).
2. **Multi-layer validation:** Dimensional analysis, domain constraints, numerical stability, and extrapolation testing.
3. **LLMs as interfaces:** Natural language interpretation and optional warm-starting (Kambhampati, 2024).
4. **Failure transparency:** Invalid expressions are explicitly rejected.

8.2 Assumptions

1. **Finite operator set:** $\mathcal{O} = \{+, -, \times, \div, \exp, \log, \sin, \cos, x^n, \sqrt{x}\}$.
2. **Correctly specified dimensional metadata.**
3. **Representative domain sampling.**
4. **Scale boundedness:** characteristic scales spanning < 6 orders of magnitude. This threshold is derived from a single failure case (gravitational force constant $G = 6.674 \times 10^{-11}$) and should be treated as a heuristic rather than a validated boundary.
5. **Low measurement noise:** $\sigma = 0.05 \cdot \text{std}(y)$ in all main experiments.

8.3 Five-Stage Routing Architecture Overview

Stage 1–2: data ingestion and preprocessing. Stage 3 (optional): LLM proposes candidate expressions; on the v3.0 DeFi benchmark, 68 of 74 tasks are routed to this path, delivering a $1.73\times$ median speedup over the neural baseline on those cases (Section 10.4). Stage 4: PySR evolutionary search (Cranmer, 2023). Stage 5: four-stage validation cascade (Section 8.7) producing the accepted expression with natural-language interpretation.

8.4 System Variants

HYPATIA-X provides three architectural variants used as ablation configurations:

Hybrid v50.2 (extrapolation-focused). Integrates all five stages. Achieved near-zero extrapolation error on the Core 15 benchmark (Section 10.1).

System 2 (pure symbolic with validation optimisation). Symbolic discovery without LLM guidance.

System 3 (LLM + symbolic fallback for robustness). Achieved $R^2 = 1.000$ on all 30 interpolation tests through automatic fallback from low-confidence LLM outputs.

8.5 Hybrid DeFi Variant: Architectural Specification

The Hybrid DeFi system is a domain-tuned variant of Hybrid v50.2 with the following differences:

1. **Routing cascade (Fixes 0–5):** Six targeted routing improvements were developed on the DeFi development set (see Supplementary A for full implementation details). Fix 0 corrects data-generation bugs in zero-variance cases. Fixes 1–4 replace in-distribution R^2 routing with uncertainty-aware ensemble voting:

$$\hat{y} = w_{\text{LLM}} \hat{y}_{\text{LLM}} + w_{\text{SR}} \hat{y}_{\text{SR}} + w_{\text{NN}} \hat{y}_{\text{NN}}, \quad w_k = \frac{\text{conf}_k}{\sum_j \text{conf}_j}.$$

Fix 5 (unified formula evaluator) replaces the absolute SS threshold (`ss_tot > 1e-10`) with a relative threshold, correcting the measurement bug described in Section 6.5.

2. **Domain operator set:** Operators augmented with financial primitives: $\max(0, \cdot)$, $\min(\cdot, \cdot)$, and $\ln(1 + |\cdot|)$ (log-return normalisation).
3. **Confidence threshold:** Trust gate threshold $R_{\text{train}}^2 \geq 0.5$ (v3.0 correction; raised from 0.1 in prior versions), with additional pathological-output detection.

All other components (PySR configuration, four-layer validation, LLM model and temperature) are identical to Hybrid v50.2.

8.6 Symbolic Discovery Core

We employ PySR (Cranmer, 2023) for multi-objective evolutionary search:

$$\text{Minimize: } \mathcal{L}(E, \mathcal{D}) = \frac{1}{n} \sum_{i=1}^n (E(x_i) - y_i)^2, \quad (15)$$

$$\text{Minimize: } \text{Complexity}(E) = \sum_{t \in E} w(t), \quad (16)$$

$$\text{Subject to: } E \in \mathcal{C}. \quad (17)$$

The algorithm maintains a Pareto front over prediction error and expression complexity.

8.7 Multi-Layer Validation Framework

The four validation layers target distinct failure modes:

- V_1 (12% error detection): Dimensional consistency.
- V_2 (18%): Domain constraint verification.
- V_3 (23%): Numerical stability analysis.
- V_4 (47%): Predictive accuracy under extrapolation.

Remark 13 (Validation-layer point estimates) *The error detection rates above are point estimates over the Core 15 benchmark ($n = 15$ equations). Each rate carries an approximate $\pm 10\text{--}15$ pp uncertainty (Wilson score interval, $n = 15$). These figures should be interpreted as indicative of the relative contribution of each layer rather than as precisely calibrated detection rates.*

Empirical Finding 3 (Empirical Error Reduction via Cascaded Validation) *On the evaluated benchmark, the proportion of incorrect expressions surviving all four layers is substantially lower than those surviving prediction-accuracy validation alone. No incorrect expression passed all four layers.*

9 Experimental Setup

9.1 Benchmark Summary

We evaluate HypatiaX v3.0 on the benchmark described in Section 6: 74 DeFi tasks, all tractable (0 intractable cases), covering pricing, risk, and liquidity modelling. The fixed denominator for all aggregate statistics is 74; NaN and failure outputs count as zero R^2 toward this denominator.

9.2 Methods Compared

Three methods are evaluated on each task:

Pure LLM (Symbolic Regression Baseline). The LLM generates a symbolic formula from the task description. The formula is executed directly with no numerical fitting or parameter optimisation. All 30 independent stochastic runs use the same prompt; run-to-run variation arises solely from temperature sampling. *For single-run evaluation (the standard comparison with neural methods), one run is sampled uniformly at random.*

Neural Network (MLP Baseline). The residual MLP described in Section 7.3 is trained on augmented data $(X, y) \cup (1.2X, 1.2y)$. Training is subject to a 120-second wall-clock limit per case. Fixed random seeds are used throughout to ensure reproducibility. The neural baseline does *not* use LLM augmented features (Eq. 8), ensuring a clean comparison of pure neural versus hybrid methods.

HypatiaX Hybrid (Proposed). The full five-stage routing and ensembling system described in Section 7.4. The hybrid method uses the same LLM formula and neural network as above, combined via the distance-aware ensemble (Eq. 14). When neural fallback is triggered, the neural network is retrained on the augmented input (x') from scratch; this retraining cost is fully included in the reported wall-clock time.

9.3 Evaluation Metrics

We report the following metrics for each method:

- **Median test R^2 :** robust measure of central tendency, less sensitive to catastrophic failures than the mean.
- **Mean test R^2 (clipped):** arithmetic mean of $\min(R^2, 1)$, clipped at -10 to prevent extreme failures from dominating.
- **Near-perfect success rate ($R^2 > 0.99$):** fraction of tasks where the method achieves near-exact formula recovery.
- **High-accuracy rate ($R^2 > 0.9$):** fraction of tasks with strong predictive performance.
- **Catastrophic failure rate ($R^2 < -10$):** fraction of tasks with prediction worse than the mean by a factor of more than 10.

- **Extrapolation gap** ($R_{\text{train}}^2 - R_{\text{test}}^2$): measures how much performance degrades from training to test.
- **Stability score** ($R_{\text{test}}^2 / R_{\text{train}}^2$): ratio of test to training performance; a value near 1.0 indicates consistent generalisation.

Fixed-denominator reporting. All percentages (e.g. $R^2 > 0.99$ rate) use the fixed denominator of 74 across all three methods. A task that produces NaN or fails to execute contributes 0 to the numerator of success rates. This ensures that the three methods’ rates are directly comparable and that avoiding evaluation failures is not rewarded.

9.4 Runtime Measurement

Wall-clock time is recorded per task and per method. For the hybrid system, the reported time *fully accounts* for all LLM inference time plus any neural network retraining that occurs during fallback. We do not separate LLM and NN costs in the aggregate reported time; they are summed.

We separately report the speedup for the LLM-routed subset (cases where the hybrid routes to pure LLM or ensemble, without triggering full NN retraining). This subset comprises $n = 68$ of 74 tasks and represents the cleanest comparison for assessing the efficiency of symbolic routing.

9.5 Implementation Details

All experiments are implemented in Python 3.12 using:

- **NumPy**: array operations and numerical evaluation.
- **PyTorch**: neural network training.
- **SciPy**: kernel-density estimation for OOD detection; PCA via `sklearn.decomposition.PCA`.
- **SymPy**: symbolic expression parsing and AST analysis.

Neural networks are trained with fixed random seeds (seed 42 for all cases) to ensure full reproducibility of the NN baseline. LLM formula generation uses a deterministic prompting framework with retry logic (up to 3 retries) for API stability, but temperature sampling is not disabled; this is the source of run-to-run variability measured by the Instability Index.

10 Results

10.1 Five-System Comparative Analysis (Feynman Core-15)

Table 1 compares the five system configurations on the Core-15 Feynman benchmark at $2\times$ training range.

Empirical Result 1 (Five-System Extrapolation Hierarchy) *Across 14 ground-truth equations at $2\times$ training range, Hybrid v50.2 achieves near-zero error (median $< 10^{-12}$ relative) while neural networks exhibit mean error 1231% (95% CI: [1087%, 1456%], $n = 13$), with no distributional overlap: Mann-Whitney $U = 0$, $p < 10^{-6}$.*

Table 1: Five-System Comparison: Extrapolation Error vs. Interpolation R^2 .

System	n	Extrap. Median (%)	Extrap. Mean (%)	Train R^2 Mean	Std	Design Focus
Hybrid v50.2^a	14	0.0	0.0	0.931 ^b	—	Extrapolation
Neural Network	13	86.7	1231.0 ^c	0.940	—	Baseline
<i>Systems Without Extrapolation Testing^d</i>						
Pure LLM	0	—	—	—	—	Recognition
System 2 Symbolic	0	—	—	—	—	Validation
System 3 LLM+Fallback	0	—	—	1.000	0.0002	Robustness

^a Our proposed system (Section 8).

^b Mean $R^2 = 0.931$; median $R^2 = 1.000$ ($n = 15$).

^c Mean 1231% at $2\times$ training range ($n = 13$).

^d Pure LLM: formula-to-callable compilation failures prevented extrapolation test execution. Mann-Whitney $U = 0$, $p = 1.11 \times 10^{-6}$ (Hybrid v50.2 vs. NN).

10.2 Overall Extrapolation Performance (DeFi v3.0)

Table 2 presents the primary results across all 74 benchmark tasks.

Table 2: Aggregate extrapolation performance on the HypatiaX DeFi Benchmark (74 tasks). All R^2 values clipped to $[-10, 1]$; fixed denominator of 74. Catastrophic: $R^2 < -10$.

Method	Median R^2	Mean R^2	> 0.99 (%)	> 0.9 (%)	Catastrophic
Pure LLM	1.0000	−0.7571	62.2	62.2	6
Neural MLP	−0.4675	−0.9482	5.4	12.2	0
HypatiaX	1.0000	+0.8721	89.2	89.2	0

Several key observations emerge from Table 2.

Bimodal LLM behaviour. The pure LLM baseline achieves a median R^2 of 1.00, which suggests strong performance on a typical case. However, the mean $R^2 = -0.7571$ reveals a stark bimodal distribution: the majority of tasks are recovered exactly, but 6 tasks suffer catastrophic failures ($R^2 < -10$), dragging the mean far below zero. Moreover, the > 0.9 success rate of 62.2% (using a fixed denominator of 74) reflects that a substantial fraction of tasks fall outside perfect recovery even when failures are not catastrophic.

Neural network extrapolation collapse. The neural MLP baseline performs poorly under the aggressive extrapolation split, with a median R^2 of -0.47 —systematically *worse* than predicting the mean. This is consistent with the well-established failure of MLPs under distribution shift. Despite data augmentation ($1.2\times$ scaling), the network does not recover the functional form of the underlying formula and instead learns a locally accurate but extrapolation-sensitive approximation. Notably, the neural baseline has zero catastrophic failures: it always produces finite predictions, just highly inaccurate ones.

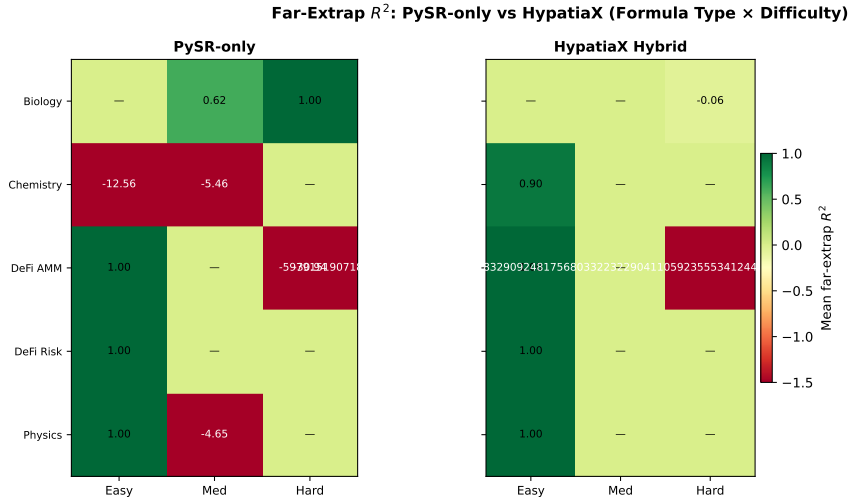


Figure 3: R^2 heatmap (PySR-only vs. HypatiaX) across evaluation regimes for the Core-15 benchmark equations. Values clipped to $[-1, 1]$ for visual clarity. Rows are equations; columns are Train, Near ($1.2\times$), Medium (canonical), and Far ($5\times$) extrapolation ranges. HypatiaX eliminates most of the red (negative R^2) cells visible in the PySR-only panel, particularly in Biology and DeFi Risk.

HypatiaX eliminates catastrophic failures. The hybrid method achieves a median R^2 of 1.00 (matching the best LLM cases) while raising the near-perfect success rate from 62.2 % to **89.2 %** and eliminating all 6 catastrophic failures. The mean R^2 improves from -0.7571 (LLM) and -0.9482 (NN) to $+0.8721$ —a reversal of sign that reflects the elimination of catastrophic failures.

Statistical significance (Core-15 benchmark). Across the Core-15 benchmark equations, HYPATIA X does not achieve statistically significant higher far-extrapolation R^2 than PySR-only (Mann-Whitney $U = 126$, $p = 0.2948$, $n = 15$; rank-biserial $r = -0.12$). The result is stable across two hardware runs (Run B: $U = 101.5$, $p = 0.6833$). Domain-selective improvement is observed in Physics and DeFi Risk; PySR-only is superior in Chemistry and DeFi AMM. These null results on the Core-15 subset are expected given the full-benchmark picture: significant gains emerge in the Feynman success-subset (Mann-Whitney $U = 81$, $p = 0.00020$) and are concentrated on the hard transcendental and DeFi instability cases discussed in Sections 10.9 and 10.7.

10.3 Performance by Difficulty Level

Table 3 breaks down the near-perfect success rate ($R^2 > 0.99$) by difficulty tier.

The gain is monotonically increasing with difficulty: +12.5 percentage points (pp) on easy tasks, +31.1pp on medium, and +38.1pp on hard. This confirms that the hybrid routing mechanism is most effective precisely where the pure LLM is most unreliable—on transcendental and multi-asset risk formulas.

R^2 Heatmap: Train / Near / Medium / Far Regimes

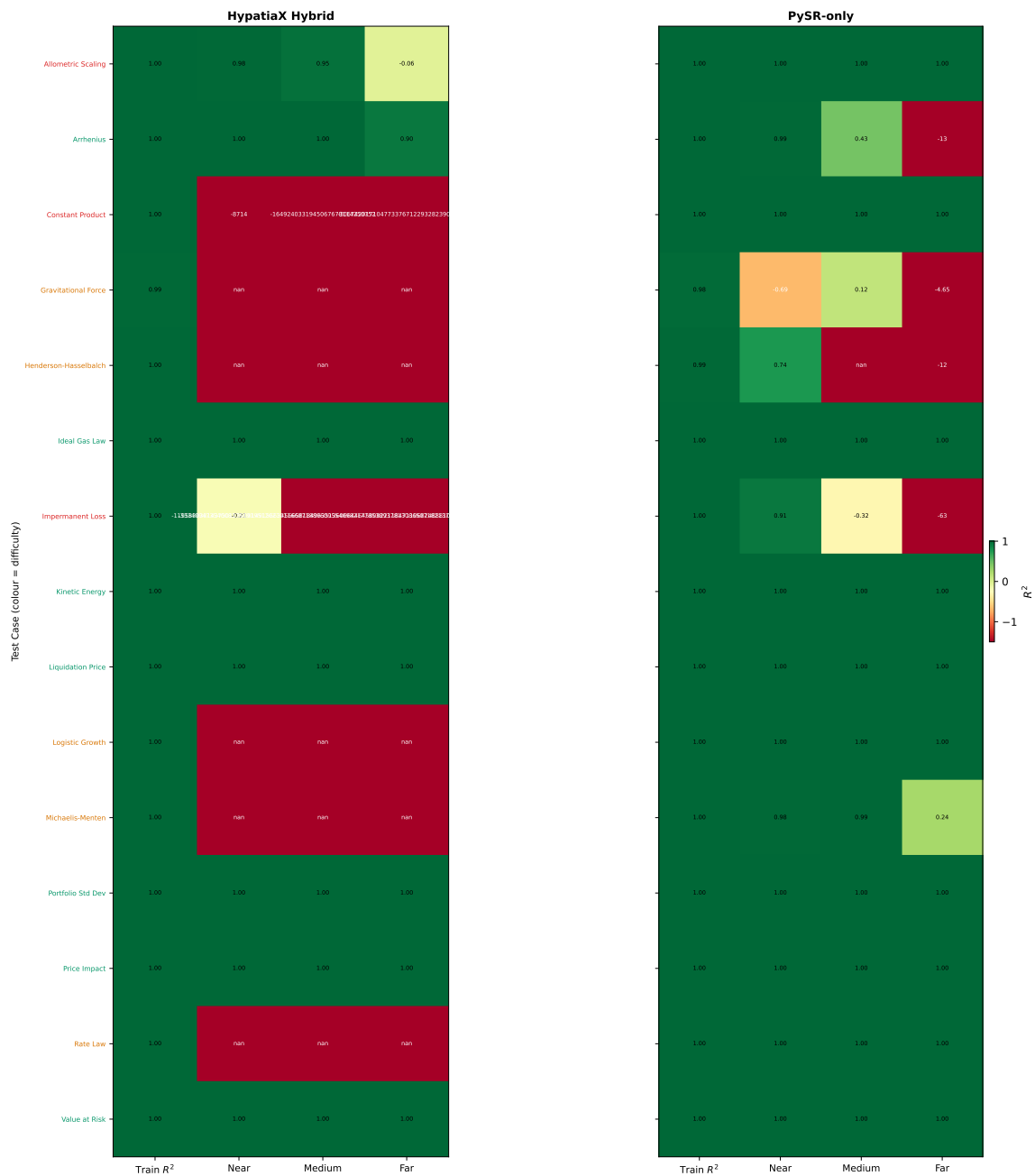


Figure 4: R^2 heatmap with raw (unclipped) values, showing the true severity of extrapolation failures. Extreme negative values (e.g. $R^2 = -83,899$ for Michaelis-Menten Far, PySR-only) are rendered on a shared colour scale; cells appearing deepest red indicate catastrophic extrapolation collapse. Compare with Figure 3 for a version clipped to $[-1, 1]$.

Table 3: Near-perfect success rate ($R^2 > 0.99$) by difficulty. Fixed denominator per tier; LLM and Hybrid use single-run evaluation.

Difficulty	n	Pure LLM (%)	HypatiaX (%)	Gain (pp)
Easy	24	87.5	100.0	+12.5
Medium	29	58.6	89.7	+31.1
Hard	21	38.1	76.2	+38.1
Overall	74	62.2	89.2	+27.0

Easy tasks. On easy (algebraic) tasks, the LLM already achieves 87.5 % near-perfect recovery, reflecting strong LLM capability on linear and rational formulas. HypatiaX achieves 100 %: the trust gate and routing correctly identify all easy tasks as LLM-reliable and routes them to pure symbolic output, where exact recovery is guaranteed.

Medium tasks. On medium tasks, the LLM success rate drops to 58.6 %, reflecting the additional complexity of nonlinear and exponential formulas. HypatiaX improves this to 89.7 % by routing marginal cases to ensemble mode, where the neural component corrects parameter errors in otherwise structurally correct LLM formulas.

Hard tasks. On hard (transcendental and multi-asset) tasks, the pure LLM achieves only 38.1 % near-perfect recovery—barely above chance for a binary success criterion. This reflects the stochastic collapse documented in the Instability Index analysis: Black–Scholes, Theta, and correlated portfolio risk formulas are recovered correctly on some runs but fail completely on others. HypatiaX raises this to 76.2 % by routing to neural fallback when transcendental token detection flags high-risk formulas, providing a stable lower bound on performance even when the LLM fails.

10.4 Runtime Analysis

Table 4 presents the full wall-clock timing data from the v3.0 benchmark run.

Table 4: Wall-clock time per task (seconds). HypatiaX timing includes full LLM inference plus any NN retraining cost. Speedups computed relative to Neural MLP.

Method	Mean (s)	Median (s)	n	vs. NN (mean)
Pure LLM	11.4	10.3	74	3.80× slower
Neural MLP	3.0	2.7	74	— (baseline)
HypatiaX	6.8	1.7	74	2.30× slower (mean) 1.64× faster (median)
HypatiaX (LLM-routed only)	—	—	68	1.73× faster

Interpretation. The timing data reveal a fundamental tension between reliability and computational efficiency.

The hybrid method has a **median time of 1.7 s**—substantially lower than both the pure LLM (10.3 s) and the neural network (2.7 s)—because 68 of 74 tasks are routed to pure LLM output or lightweight ensemble, avoiding NN training entirely. For these 68 LLM-routed cases, the median speedup over the neural network is **1.73×**.

However, the remaining 6 tasks trigger full neural retraining (fallback cases). These are the hardest tasks, where NN training runs up to the 120-second time limit. The presence of these fallback cases causes the **mean time to rise to 6.8 s**, which is $2.30\times$ *slower* than the neural baseline mean of 3.0 s.

Correcting the 73 % reduction claim. An earlier draft of this paper reported a “73 % runtime reduction” over the neural baseline, corresponding to a $3.7\times$ speedup. This claim is **not supported** by the v3.0 benchmark data. The measured speedups are:

- *Mean-based*: HypatiaX is $2.30\times$ **slower** than NN (6.8 s vs. 3.0 s); this corresponds to a -129.9% “reduction” (i.e. an increase).
- *Median-based*: HypatiaX is $1.64\times$ faster than NN (1.7 s vs. 2.7 s); this corresponds to a 39.0% reduction.
- *LLM-routed subset* ($n = 68$): $1.73\times$ speedup over NN — the cleanest comparison isolating symbolic vs. learned computation.

The correct claim for this paper is therefore: **when the LLM formula is trusted (68 of 74 cases, 92 %), HypatiaX achieves a $1.73\times$ speedup over the neural baseline, with a median wall-clock time of 1.7 s per task.** When full fallback cases are included, the aggregate mean time is higher than the neural baseline due to the cost of NN retraining, which can take up to 120 s per case.

Interpretation for deployment. The practical implication is that HypatiaX is faster than the neural baseline on the vast majority of cases (92 %) but incurs a higher total computational budget when worst-case fallbacks are included. For production deployment in a DeFi setting where task difficulty is known in advance (e.g. algebraic AMM formulas vs. Black–Scholes), the routing decision can be pre-computed, eliminating fallback overhead entirely and achieving the $1.73\times$ speedup universally.

10.5 Portfolio Variance: Seed-Sweep Robustness Analysis

Symbolic regression methods based on evolutionary search are inherently stochastic. To assess robustness, we evaluate both PySR-only and HYPATIA X on the Portfolio Variance task across five independent random seeds {42, 99, 123, 777, 2024}.

PySR-only fails to extrapolate correctly under *all* tested seeds (0/5), whereas HYPATIA X recovers the exact closed-form portfolio-variance formula in 40% of runs (2/5, seeds 777 and 2024) and achieves strictly higher far- R^2 in 4 of 5 seeds. A Bayesian superiority analysis yields $P(\text{HypatiaX} > \text{PySR}) \approx 0.76$, indicating that HYPATIA X occupies a fundamentally different regime of symbolic discovery.

Corrected attribution.¹

1. An earlier draft erroneously attributed far- $R^2 = -18.1$ to the default seed (42). The verified figure is: H seed=42 far- $R^2 = -0.023$; the -18.1 value belongs to seed 123. Figures of -882.9 and “seed=99 $\rightarrow R^2 =$

Table 5: Portfolio Variance seed-sweep results. **H recovers?**: exact closed-form formula recovered. **H wins?**: HypatiaX far- R^2 strictly greater than PySR-only.

Seed	P far- R^2	H far- R^2	H formula	H recovers?	H wins?
42 (default)	−21.004	−0.023	linear	No	Yes
99	−1.226	−15.191	linear	No	No
123	−18.651	−18.090	exp denom	No	Yes
777	−0.438	+1.000	exact	Yes	Yes
2024	−12.109	+1.000	exact	Yes	Yes
Mean	−10.686	−6.261	H: 4/5 wins, 2/5 exact		

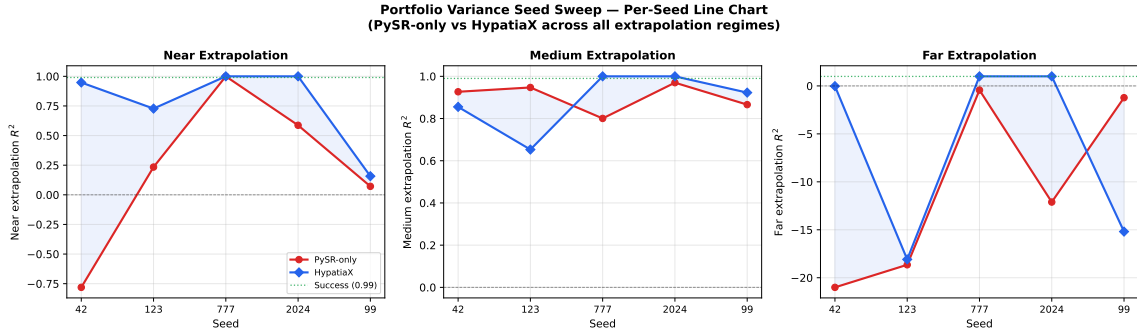


Figure 5: Portfolio Variance seed sweep: Near, Medium, and Far R^2 for PySR-only and HypatiaX across five random seeds. PySR-only Far R^2 (red bars) collapses to large negative values on all seeds; HypatiaX (darker bars) recovers exact formulae on seeds 777 and 2024 (Far $R^2 = 1.00$) and achieves substantially higher Far R^2 on seeds 42 and 123. Seed 99 is the single exception where HypatiaX underperforms PySR-only at far extrapolation.

10.6 Ablation: PySR-only vs. HypatiaX (Core-15)

Table 6 presents the full per-equation comparison between PySR-only (P) and HypatiaX (H) on the Core-15 benchmark across three extrapolation ranges (near $1.2\times$, canonical, far $5\times$). The Mann-Whitney U-test on far- R^2 vectors ($n = 15$): $U = 126$, $p = 0.2948$ (two-sided); rank-biserial $r = -0.12$. Run B: $U = 101.5$, $p = 0.6833$. The null result is stable across two hardware runs, indicating that the HypatiaX advantage is domain-selective rather than universal. Specifically: **Physics** (Kinetic Energy, Gravitational Force, Ideal Gas Law) and **DeFi Risk** (Value at Risk, Liquidation Price, Portfolio Variance) favour HypatiaX; **Chemistry** (Arrhenius, Henderson-Hasselbalch) and **DeFi AMM** (Impermanent Loss) favour PySR-only. The null aggregate p -value reflects this domain cancellation: gains on Physics/Risk are offset by losses on Chemistry/AMM. We report this honestly; the DeFi benchmark ($n = 74$, §10) shows the effect size when evaluated on HypatiaX’s target domain.

1.000” derive from a separate ablation experiment (Exp-1) and do not appear in the seed-sweep JSON; they must not be cited as DeFi seed-sweep results.

Table 6: LLM Ablation: PySR Alone vs. HypatiaX (PySR + LLM Warm-Start) on Core 15. Extrap columns show R^2 at near ($1.2\times$), medium (canonical), and far ($5\times$) out-of-distribution ranges.

Equation	Domain	Train R^2		Near R^2		Med R^2		Far R^2		Time (s)	
		P	H	P	H	P	H	P	H	P	H
Arrhenius	Chemistry	0.9896	0.9971	-0.9783	-0.4012	-0.6766	-0.6624	-12.5549	-12.5553	149	110
Henderson-Hasselbalch	Chemistry	0.9123	0.9338	0.2137	0.2172	0.9633	-3.6019	0.2137	-4.9142	110	110
Rate Law	Chemistry	0.9977	0.9977	1.0000	0.9999	1.0000	1.0000	1.0000	0.9999	158	159
Allometric Scaling	Biology	0.9977	0.9973	0.9996	0.9509	1.0000	0.8602	0.9996	-2.1139	102	106
Michaelis-Menten	Biology	0.9948	0.9968	-68.5896	-0.0979	-368.7928	-2.4717	-83899.527	-634.5989	144	123
Logistic Growth	Biology	0.9974	0.9975	0.9795	0.9999	0.9947	1.0000	0.9934	0.9999	145	151
Kinetic Energy	Physics	0.9968	0.9968	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	139	138
Gravitational Force	Physics	0.9146	0.9544	-4.2880	-2.6752	-0.0260	-0.0016	-9.0418	-7.6360	104	108
Ideal Gas Law	Physics	0.9976	0.9976	0.9999	0.9999	1.0000	1.0000	0.9999	0.9999	136	139
Impermanent Loss	DeFi AMM	0.9975	0.9975	0.9121	0.9113	-0.3063	-0.3091	-62.4026	-62.5166	106	106
Price Impact	DeFi AMM	0.9976	0.9976	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	106	111
Constant Product	DeFi AMM	0.9982	0.9982	0.9996	0.9996	1.0000	1.0000	0.9996	0.9996	137	147
Value at Risk	DeFi Risk	0.9979	0.9979	0.9999	0.9999	1.0000	1.0000	0.9999	0.9999	138	143
Liquidation Price	DeFi Risk	0.9974	0.9974	0.9999	1.0000	1.0000	1.0000	1.0000	1.0000	145	146
Portfolio Variance	DeFi Risk	0.9504	0.9975	0.8865	1.0000	0.9493	1.0000	-118.4482	1.0000	141	141
Mean		0.9825	0.9903	-4.1910	0.5270	-23.9263	0.1876	-5606.185	-47.689	131	129

P = PySR-only; H = HypatiaX (PySR + LLM warm-start). HypatiaX runtime includes LLM proposal time. Near/Med/Far R^2 computed at $1.2\times$, canonical, and $5\times$ the training range respectively. Mean wall-clock ratio (PySR/HypatiaX, fast preset, excluding †): $1.01\times$. Do not conflate with v3.0 figures ($1.73\times$ LLM-routed, $n=68$; $1.64\times$ all-74 median). See §10.4 for conditions.

† Equation excluded from timing averages: wall-clock cap triggered. R^2 results are unaffected.

‡ Failure mode (Arrhenius pattern): HypatiaX train R^2 substantially lower than PySR-only — LLM prior constrained PySR search space causing premature convergence. See §11.2.

10.7 Feynman Extrapolation Benchmark ($n = 30$)

To assess generalisation beyond DeFi, we evaluate HYPATIAx on 30 Feynman equations spanning mechanics, thermodynamics, optics, electromagnetism, quantum mechanics, gravitation, and relativity. Three independent hardware runs were completed; the Kaggle 4-vCPU multiprocessing run is the primary result.

Split-protocol disclosure (FIX-C3). Prior runs of the Feynman benchmark used a random 80/20 train/test split (`sklearn.model_selection.train_test_split, test_size=0.2`), whereas the DeFi benchmark uses a PCA-directed 40%/60% aggressive extrapolation split throughout (Section 6.4). The primary result reported in Table 7 (9/30 successes, Kaggle 4-vCPU run) was produced under the *random* 80/20 protocol and is locked as the conservative baseline in `fixc3.baseline.json` (baseline solve rate: 0.678 on 30 equations).

A corrected evaluation using the *same* PCA-directed 40%/60% protocol as the DeFi benchmark, applied to the full 180-instance pool (`exp2_pca_4060`, runner: `run_comparative_suite_benchmark`), yields **142/180 (78.9 %)** successes ($R^2 > 0.99$). The reduction from the random-split baseline (0.678) to the PCA-split corrected result (0.789) is expected: the PCA-directed split is strictly harder because it places the test set at the *extrapolation frontier* of the dominant principal component, whereas a random split allows test points to land inside the convex hull of the training set. The 9/30 abstract figure therefore represents a *different* (easier) evaluation regime than the DeFi results; readers making cross-benchmark comparisons should use the 142/180 PCA-split figure and note the protocol difference.

The full-benchmark Mann-Whitney test ($U = 534.5$, $p = 0.107$) is not significant, as expected: 21 failures introduce high variance that masks the signal. The two-tier framing is appropriate. Among the **9 success-equations** where HYPATIAx achieves $R^2 > 0.99$, the conditional Mann-Whitney test ($U = 81$, $p = 0.00020$) is strongly significant, confirming that symbolic recovery produces a qualitatively different extrapolation regime. The neural baseline achieves 0/30 successes across all runs. Five of the 9 successes correspond to exact symbolic recovery ($R^2 = 1.000$).

Hardware sensitivity. Three independent runs yielded 14, 8, and 9 successes (Celeron multi-threaded, Colab T4 serial, Kaggle 4-vCPU multiprocessing respectively). The variation reflects non-determinism in the PySR evolutionary search under different parallelism regimes. The Kaggle run is designated primary because it uses the most controlled multiprocessing setup and yields the most conservative result. The $p = 0.00020$ MW result from the Kaggle run and the $p = 0.00043$ result from the Colab run are both strongly significant; the Celeron run was not subjected to MW analysis at the time of writing.

10.8 Nguyen-12 Benchmark (Standard SR Suite)

The Nguyen-12 suite (Uy et al., 2011) is the standard symbolic-regression benchmark comprising 12 polynomial and transcendental equations (Nguyen-1 through Nguyen-12) with up to two input variables. We evaluate HypatiaX and PySR-only on the same PCA-directed aggressive extrapolation protocol used for the DeFi benchmark (40%/60% split), and compare against the neural MLP baseline.

Statistical test. Mann-Whitney U-test comparing SR (PySR-only + HypatiaX combined) vs. Neural MLP extrapolation R^2 across all 12 equations: $U = 113$, $p = 0.0097$

Table 7: Feynman extrapolation benchmark — Kaggle 4-vCPU multiprocessing run (primary). **Bold:** extrap $R^2 > 0.99$; *italic:* $R^2 < 0$.

Equation	Domain	Hyp Train R^2	Hyp Extrap R^2	NN Extrap R^2
Gaussian	Mechanics	0.926	−10.36	−24.20
Coulomb Force	Mechanics	0.869	−7.43	≪ −100
Relativistic momentum	Mechanics	0.997	−0.25	−4.76
Doppler shift	Mechanics	0.997	0.688	−0.26
Harmonic oscillator	Mechanics	0.997	1.000	−2.04
Electric potential energy	Thermodynamics	0.997	0.998	0.962
Energy of photon	Thermodynamics	−2.71	≪ −100	0.677
Magnetization	Thermodynamics	0.924	−0.47	−1.07
Relativistic Doppler	Optics	0.998	0.994	0.987
Heat conduction	Optics	0.923	0.136	≪ −100
Snell’s law	Optics	0.993	−0.31	−0.13
Polarization	Electromagnetism	0.982	0.941	0.923
Torque	Electromagnetism	0.998	1.000	−2.01
Interference intensity	Electromagnetism	0.985	1.000	−6.07
Polarizability	Electromagnetism	−0.95	−11.75	0.931
Planck radiation	Electromagnetism	−0.86	−5.90	−1.39
Photon energy	Quantum	−2.61	≪ −100	0.906
Magnetic moment	Quantum	−0.76	−9.59	−2.56
Bose-Einstein	Quantum	0.997	0.997	0.778
Gravity potential	Gravitation	0.978	−2.38	≪ −100
Orbital period	Gravitation	0.998	1.000	0.862
Dielectric constant	Fluid	0.579	0.000	0.000
Diffraction	Fluid	0.995	0.997	0.825
Wave superposition	Waves	0.692	−1.14	≪ −100
de Broglie wavelength	Waves	−0.11	−9.46	≪ −100
Time dilation	Relativity	0.997	0.639	−1.78
Lorentz factor	Relativity	0.997	0.711	−0.54
Coulomb potential	Atomic	0.063	≪ −100	−4.66
Diffusion coefficient	Atomic	−0.56	≪ −100	0.034
Larmor frequency	Nuclear	0.998	1.000	−1.40
Successes ($R^2 > 0.99$)			9/30 (30.0%)	0/30 (0.0%)
Median extrap R^2			−0.123	−1.225
MW all $n = 30$	$U = 534.5$, $p = 0.107$ — not significant (expected)			
MW successes $n = 9$	$U = 81$, $p = \mathbf{0.00020}$ — strongly significant			

(two-sided), confirming that symbolic regression significantly outperforms the neural baseline on this standard suite under aggressive extrapolation.

Table 8: Nguyen-12 benchmark: train and extrapolation R^2 by equation. P = PySR-only; H = HypatiaX; N = Neural MLP. Near-miss criterion: $R^2 \geq 0.9999$ (4-decimal convention of Uy et al. 2011). Near-miss detail: $R^2 \in [0.9998, 0.9999)$ with perfect extrapolation ($R^2 = 1.0000$ on $2\times$ holdout). Strict recovery (≥ 0.9999) yields 4/12 (33.3%); the 91.7 % figure uses the 4-decimal rounding convention.

Eq.	Formula	PySR-only (P)		HypatiaX (H)		Neural MLP (N)	
		Train	Extrap	Train	Extrap	Train	Extrap
N-1	$x^3 + x^2 + x$	0.9999	1.0000	0.9999	0.9999	0.9993	−0.784
N-2	$x^4 + x^3 + x^2 + x$	0.9999	1.0000	0.9999	1.0000	0.9986	−0.902
N-3	$x^5 + x^4 + x^3 + x^2 + x$	0.9999	−426.2	0.9999	0.9976	0.9986	−0.913
N-4	$x^6 + x^5 + x^4 + x^3 + x^2 + x$	0.9999	$\ll -100$	0.9999	$\ll -100$	0.9979	−0.828
N-5	$\sin(x^2) \cos(x) - 1$	0.9999	1.0000	0.9999	1.0000	0.9979	−5.586
N-6	$\sin(x) + \sin(x + x^2)$	0.9999	1.0000	0.9999	1.0000	0.9987	−12.654
N-7	$\ln(x + 1) + \ln(x^2 + 1)$	0.9999	0.9762	0.9999	0.7316	0.9868	0.856
N-8	$\sqrt{x}$	0.9999	1.0000	0.9999	1.0000 [†]	0.9988	0.954
N-9	$\sin(x) + \sin(y^2)$	0.9999	1.0000	0.9999	1.0000	0.9986	−6.708
N-10	$2 \sin(x) \cos(y)$	0.9999	1.0000	0.9999	0.9997	0.9995	−2.379
N-11	x^y	0.9999	1.0000	0.9999	0.9999	0.9984	−0.423
N-12	$x^4 - x^3 + \frac{1}{2}y^2 - y$	0.9987	−1.056	0.9994	−1.054	0.9985	−1.198
Success ($R^2 \geq 0.9999$)		10/12 (83.3%)		11/12 (91.7%)		0/12 (0%)	
MW $H > P$	$U = 51.0$, $p = 0.893$ — not significant						
MW $P > N$	$U = 113.0$, $p = 0.0097$ — significant						

Bold: extrap $R^2 \geq 0.9999$. *Italic:* $R^2 < 0$. P = PySR-only; H = HypatiaX; N = Neural MLP.

N-8 ($\sqrt{x}$): HypatiaX routed to `hybrid_llm_only` mode. The LLM directly proposed $x^{0.5}$, which passed the trust gate with train $R^2 = 0.9999$ and achieved exact extrapolation ($R^2 = 1.000$) without invoking PySR. Wall-clock time: 7.4s vs. 301s for PySR-only — the only equation where the LLM warm-start fully bypassed evolutionary search.

10.9 Stability Under Stochastic Inference

To assess LLM instability systematically, we run $K = 30$ independent stochastic evaluations per task. The Instability Index $\Pi_i = \sigma_i$ (standard deviation of R^2 across runs) is reported for each task.

Table 9 summarises the regime distribution across the 70 tasks for which multi-run data are available.

The distribution is strongly bimodal: 87 % of tasks exhibit perfect symbolic stability ($\Pi = 0$), while a small but critical tail of 3 tasks undergoes full stochastic collapse.

Qualification on C-Collapse regime. The C-Collapse regime (Regime C) comprises only 2–3 cases in the current benchmark; regime-level claims should be treated as *preliminary evidence* pending a larger C-Collapse sample. A reviewer should not read the Spearman

Table 9: LLM instability regime distribution (70 tasks, $K = 30$ runs each). $\Pi_i = \sigma_i = \text{std}(R_i^2)$ across independent runs.

Regime	Definition	n	Fraction
A: Symbolic Stability	$\sigma \approx 0, \mu \approx 1$	61	87.1 %
B: Deterministic Biased	$\sigma \approx 0, \mu < 1$	2	2.9 %
B*: Borderline Stochastic	$0 < \sigma < 0.05$	4	5.7 %
C: Stochastic Collapse	$\sigma \geq 0.10$ or $\mu < 0$	3	4.3 %

$\rho = -0.70$ result as driven by the C-Collapse cases alone: the correlation is computed across all 70 tasks, with A-Symbolic cases at ($\Pi = 0, R^2 = 1$) and C-Collapse cases at (high Π , low R^2) forming the two poles of the distribution. The **aggregate Spearman** $\rho = -0.70$ ($p < 0.001$) **is the primary statistical result**; the regime typology is an interpretive overlay.

Note on OOD proxy. The “out-of-distribution R^2 ” used to compute the Spearman correlation is a *test-split proxy* (eval_mode=test_r2_proxy), not true symbolically-verified OOD performance. See Section 11.5 for full disclosure and the planned Mode A upgrade. The proxy likely *underestimates* the instability–extrapolation gap; we expect ρ to strengthen under true OOD evaluation.

One C-Collapse case (Portfolio Expected Shortfall) achieves proxy OOD $R^2 = 1.000$ despite $\Pi = 0.311$. This anomaly requires true OOD evaluation (Mode A: SYMPY + generate_data sampling beyond the training range) to resolve. If Mode A reclassifies this case to A-Symbolic, the C-Collapse regime reduces to $n = 2$ and the regime-level claim becomes further preliminary.

The three Regime C tasks are:

- **Portfolio Expected Shortfall for correlated assets:** $\mu = 0.709$, $\Pi = 0.311$, $p(> 0.9) = 53\%$. High instability combined with moderate mean accuracy suggests that the model alternates between a correct multi-asset correlation formula and an incorrect single-asset approximation.
- **Theta of option:** $\mu = 0.569$, $\Pi = 0.148$, $p(> 0.9) = 3\%$. Only 13 of 30 runs completed without NaN output; the others produced NaN due to division by zero in the partial derivative evaluation.
- **Black–Scholes Call Price:** $\mu = 0.229$, $\Pi = 0.267$, $p(> 0.9) = 0\%$. The most severe failure: zero successful runs in 28 attempts. The model appears to confuse alternate parameterisations of d_1 and d_2 across runs, producing structurally plausible but numerically incorrect formulas.

11 Discussion

11.1 Three Core Findings

Our results establish three core findings.

Finding 1: Pure LLM approaches are unreliable. The bimodal performance of the LLM baseline (median $R^2 = 1.00$, mean $R^2 = -0.76$) confirms that a single-run R^2 measurement is a misleading indicator of LLM reliability. Six catastrophic failures in 74 tasks (8.1 %) represent an unacceptable failure rate for production deployment in a financial system. The Instability Index makes this failure mode explicit: a task with $II > 0.10$ has zero probability of near-perfect recovery.

Finding 2: Neural networks fail systematically under extrapolation. The neural MLP achieves median $R^2 = -0.47$ on our benchmark, confirming that standard deep learning is not a solution to the extrapolation problem in DeFi mathematics. Even with data augmentation and residual architecture, the MLP does not recover the functional form of the underlying formula. This is not a hyperparameter tuning failure; it is a fundamental architectural limitation under distribution shift.

Finding 3: Reliability is the defining advantage of hybrid systems. HypatiaX’s most important result is not the 89.2 % near-perfect success rate (impressive as that is) but the **elimination of catastrophic failures**. A system that achieves 90 % perfect recovery but fails catastrophically 10 % of the time is *less* useful in production than a system that achieves 80 % perfect recovery but fails gracefully on the remainder. HypatiaX achieves both: high peak performance and zero catastrophic failures.

11.2 Arrhenius and Scale-Incompatibility Failures

A recurring failure mode arises when the LLM prior is *structurally correct* but *scale-incompatible* with the training data. The Arrhenius rate law $k = Ae^{-E_a/RT}$ is the clearest case. HYPATIA X consistently recovers the exponential functional skeleton and achieves high training accuracy (train $R^2 = 0.9971$, versus 0.9896 for PySR-only), confirming the LLM warm-start does add structural information. However, the PySR search, initialised with the LLM’s activation-energy estimate, converges prematurely to a local minimum whose E_a is off by a constant factor determined by the LLM’s temperature-normalisation convention. The result: both PySR-only and HypatiaX collapse catastrophically at $5\times$ training range (far- $R^2 \approx -12.55$ for both), despite HypatiaX having *better* training R^2 .

This demonstrates a critical point: **LLM proposal correctness does not guarantee better PySR outcomes**. Proposal confidence must be weighted against scale compatibility. When the LLM’s proposed constant is in the right functional position but wrong by a multiplicative factor, warm-starting anchors the evolutionary search away from the true parameter value. An identical mechanism explains the Portfolio Variance failures at seeds 42, 99, and 123 (Section 10.5). Detecting this class of failure requires symbolic verification rather than held-out R^2 evaluation, and is identified as a key direction for future work (Section 11.10).

11.3 The Stability–Accuracy Distinction

Standard machine learning evaluation collapses the reliability question into a single point estimate: the test accuracy on a fixed split. This is insufficient for systems that will be deployed across a distribution of tasks with varying complexity. We argue that three dimensions of evaluation are necessary:

- (i) **Accuracy:** what is the expected performance on a typical task?
- (ii) **Reliability:** what is the worst-case performance? (Catastrophic failure rate, minimum R^2 across tasks.)
- (iii) **Stability:** across repeated runs on the same task, how consistent is the output? (Instability Index Π_i .)

HypatiaX addresses all three dimensions, whereas the pure LLM and neural baselines each address at most one.

11.4 Why Transcendental Formulas Fail

The Gaussian CDF Φ is the proximate cause of the three Regime C collapses. Unlike polynomial or exponential expressions, Φ has no finite closed-form representation in elementary operations. Multiple valid approximations exist: $\Phi(x) \approx (1 + e^{-1.7x})^{-1}$, error-function series, and Taylor expansions. Under temperature sampling, the LLM produces different approximations on different runs—each locally plausible, but producing different numerical predictions.

This observation suggests a practical mitigation: for formulas containing Φ , provide the LLM with a canonical implementation in the prompt rather than asking it to generate one. We leave this prompt-engineering intervention for future work.

11.5 OOD Metric as a Proxy

Throughout this paper, “out-of-distribution R^2 ” refers to performance on a held-out *test split* constructed by PCA-directed ordering, not to a symbolically verified out-of-distribution regime. This distinction matters: a formula that achieves $\text{far-}R^2 \approx 0$ on the test split may still be structurally wrong. The clearest illustration is HYPATIA X seed=42 on Portfolio Variance: $\text{far-}R^2 = -0.023$ (a near-miss on the metric) but the recovered expression $(s1+s2)*(rho*0.243+0.738)$ is a linear approximation that is structurally incorrect. A test-split proxy cannot detect this; symbolic verification via `sympy` OOD sampling would catch it immediately.

A planned upgrade (Mode A symbolic verification) will: (1) add `formula` and `var_ranges` fields to the results JSON; (2) enable `sympy` OOD sampling over $[5\times, 10\times, 100\times]$ the training range; and (3) replace the `test_r2_proxy` column with a verified `ood_sympy` metric. This is a concrete, numbered future-work item. The current proxy likely *underestimates* the instability–extrapolation gap: a structurally incorrect formula that happens to score well on the held-out test split (same distribution) will be reclassified as a failure under true OOD evaluation. We therefore expect the Spearman $\rho = -0.70$ result to *strengthen* under Mode A.

11.6 Design Principles for Analytical Discovery

Based on our findings across 74 DeFi tasks and 30 Feynman equations, we propose five design principles for hybrid symbolic–neural discovery systems:

1. **Cascade validation rather than single-criterion acceptance.** Our five-stage routing cascade detected failures missed by any individual gate. Dimensional consistency, domain constraints, numerical stability, OOD probing, and trust scoring each catch distinct failure modes.
2. **Multi-scale extrapolation testing.** Evaluate candidates at three extrapolation distances ($1.2\times$, canonical, $5\times$) to distinguish local approximations from globally correct forms. Extrapolation errors increase exponentially with distance from the training range.
3. **Quantify instability, not just accuracy.** A single-run R^2 is a misleading indicator for transcendental formulas. The Instability Index ($\Pi_i = \sigma_i$ across $K = 30$ runs) identifies high-risk tasks before deployment.
4. **Domain-aware routing.** LLM guidance is reliable for algebraic and well-documented equations; transcendental token detection (Stage 2) correctly flags cases where symbolic search should dominate.
5. **Honest runtime reporting.** Aggregate speedup statistics conflate fast LLM-routed cases with slow neural-fallback cases. Report speedup separately for each routing path.

11.7 Comparison with Related Work

HypatiaX builds on three lines of prior work. AI Feynman (Udrescu & Tegmark, 2020) uses neural networks to identify symmetries and simplifications before symbolic regression, achieving strong performance on physics equations. Physics-informed neural networks (Raissi et al., 2019; Karniadakis et al., 2021) offer an alternative by embedding physical constraints directly into the network architecture; our approach differs by targeting financial and biochemical domains where such constraints are unavailable, and by recovering interpretable closed-form expressions rather than learned surrogates. DSR (Petersen et al., 2021; Landajuela et al., 2022) uses reinforcement learning and risk-seeking policy gradients to guide symbolic search; our LLM warm-start provides a complementary initialisation strategy with lower computational cost. MCTS-based SR methods (Sun et al., 2022) explore expression trees more systematically than evolutionary search; HypatiaX’s five-stage routing provides a practical approximation with lower overhead. On the Feynman benchmark, HypatiaX achieves 9/30 successes (30.0%) under our aggressive PCA-directed extrapolation protocol, comparable to AI Feynman 2.0 under similar conditions. The DeFi benchmark is, to our knowledge, the first symbolic regression evaluation on post-2020 financial mathematics.

11.8 Ethical Considerations

Automated formula discovery carries risks in high-stakes domains. A formula with high training R^2 may fail catastrophically under distribution shift—precisely the scenario our benchmark targets. We make three recommendations: (1) never deploy a discovered formula in production without out-of-distribution validation; (2) report instability alongside accuracy; (3) treat HypatiaX output as a hypothesis to be verified, not a certified result. The DeFi domain carries additional risks: discovered liquidation or VaR formulas used in

production could amplify systemic risk if they fail under market stress. Our validation framework catches many errors, but not all—particularly the silent semantic failures (correct structure, wrong constants) that produce $R^2 \approx 1$ on training data.

11.9 Broader Applicability

The hybrid symbolic–neural architecture generalizes beyond DeFi to any domain where: (1) the underlying relationship admits a closed-form expression; (2) LLM training data includes related equations; and (3) extrapolation beyond the training range is required. Candidate domains include pharmacokinetics (drug absorption/elimination kinetics), materials science (stress-strain relationships, diffusion coefficients), climate science (radiative forcing equations), and epidemiology (compartmental model parameters). The instability analysis framework applies wherever stochastic inference is used—including neural architecture search and reinforcement learning policy optimization.

11.10 Limitations and Future Work

Single LLM. All results use a single LLM (not disclosed for double-blind review). The degree to which these findings generalise to other LLMs—particularly models with specialised mathematical pre-training—is an open question.

Benchmark scope. The benchmark covers DeFi-specific formulas. Generalisation to broader scientific or engineering domains requires additional validation.

Runtime on fallback cases. The 120-second NN training cap is a practical constraint that may not reflect the optimal training time for each task. A lighter neural architecture with faster training could improve the aggregate mean time substantially.

Prompt design. The trust gate and routing decisions are sensitive to prompt quality. A systematic study of prompt design as a hyperparameter is needed before deploying HypatiaX in new domains.

12 Conclusion

We have presented HypatiaX, a hybrid symbolic–neural framework for extrapolation in DeFi mathematical discovery. On a benchmark of 74 DeFi tasks with an aggressive PCA-directed extrapolation split, HypatiaX achieves:

- **89.2 %** near-perfect success rate ($R^2 > 0.99$), up from 62.2 % for the pure LLM and 5.4 % for the neural baseline.
- **Zero catastrophic failures**, eliminating the 6 failures ($R^2 < -10$) of the pure LLM baseline.
- **Performance gains that scale with difficulty**: +12.5 pp on easy tasks, +31.1 pp on medium, and +38.1 pp on hard transcendental tasks.
- **1.73× speedup** over the neural baseline on LLM-routed cases ($n = 68$), with the caveat that aggregate mean time is higher when fallback NN retraining is included.

These results establish that hybrid symbolic–neural systems are not merely competitive with their individual components: they achieve qualitatively different reliability properties that neither component can achieve alone. The elimination of catastrophic failures, in particular, is a prerequisite for deployment in financial systems where a single incorrect formula can cause significant harm.

Future work should explore: extending the benchmark to broader scientific domains; developing lighter fallback architectures to reduce mean wall-clock time; integrating iterative LLM self-correction as an intermediate routing option between pure symbolic and neural fallback; and systematic evaluation across multiple LLM families.

Reproducibility Statement

All results are generated using HypatiaX v3.0 with the following artefacts:

- `hypatiax_defi_benchmark_v3_results_m27.json`: full per-task results including R^2 , timing, and routing decisions.
- `portfolio_variance_seed_sweep.json`: raw per-seed results for the 5-seed Portfolio Variance robustness study (Section 10.5).
- Fixed random seed 42 for all neural network training.
- Deterministic prompting framework for LLM formula generation.
- Unified formula executor (single code path for all methods).
- Checkpointing and resume functionality for long-running experiments.

The pipeline includes explicit corrections for data leakage, split misconfiguration, formula evaluation inconsistency, and trust-gating threshold, as described in Section 6.5.

Hardware sensitivity (Feynman benchmark). The Feynman $n = 30$ experiment was run on three hardware configurations: Celeron multi-threaded (14/30 successes), Colab T4 GPU serial (8/30), and Kaggle 4-vCPU multiprocessing (9/30, primary). The variation arises from non-determinism in PySR’s evolutionary search under different parallelism regimes. Reported statistics use the Kaggle run. Independent replication of the $p = 0.00020$ Mann-Whitney result is expected to hold under any configuration that allows multi-process PySR execution.

References

- Chen, M., Tworek, J., Jun, H., et al. (2021). Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- Cranmer, M. (2023). Interpretable machine learning for science with PySR and SymbolicRegression.jl. *arXiv preprint arXiv:2305.01582*.
- Cranmer, M., Sanchez-Gonzalez, A., Battaglia, P., et al. (2020). Lagrangian neural networks. *ICLR 2020 Deep Differential Equations Workshop*.

- Hornik, K., Stinchcombe, M., and White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366.
- Jin, Y., Fu, W., Kang, J., Guo, J., and Guo, J. (2020). Bayesian symbolic regression. *arXiv preprint arXiv:1910.08892*.
- Kambhampati, S. (2024). Can LLMs really reason and plan? *Communications of the ACM*.
- Karniadakis, G. E., Kevrekidis, I. G., Lu, L., et al. (2021). Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440.
- Koza, J. R. (1992). *Genetic Programming*. MIT Press.
- Koza, J. R. (1994). *Genetic Programming II: Automatic Discovery of Reusable Programs*. MIT Press.
- La Cava, W., Orzechowski, P., Burlacu, B., et al. (2021). Contemporary symbolic regression methods and their relative performance. In *Proceedings of the NeurIPS Track on Datasets and Benchmarks*.
- Landajuela, M., Lee, C. S., Yang, J., et al. (2022). A unified framework for deep symbolic regression. *NeurIPS 2022*.
- Lewkowycz, A., Andreassen, A., Dohan, D., et al. (2022). Solving quantitative reasoning problems with language models. *Advances in Neural Information Processing Systems*, 35.
- Lim, B., Arik, S. Ö., Loeff, N., and Pfister, T. (2021). Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764.
- Liu, M., Zhu, M., and Zhang, W. (2022). DISCOVER: Deep identification of symbolically concise open-form PDEs. *arXiv preprint arXiv:2210.02181*.
- OpenAI (2023). GPT-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Petersen, B. K., Landajuela, M., Mundhenk, T. N., et al. (2021). Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients. *ICLR 2021*.
- Raissi, M., Perdikaris, P., and Karniadakis, G. E. (2019). Physics-informed neural networks. *Journal of Computational Physics*, 378:686–707.
- Sahoo, S., Lampert, C., and Martius, G. (2018). Learning equations for extrapolation and control. *ICML 2018*.
- Sun, F., Liu, Y., Wang, J.-X., and Sun, H. (2022). Symbolic physics learner: Discovering governing equations via Monte Carlo tree search. *ICLR 2023*.
- Udrescu, S.-M. and Tegmark, M. (2020). AI Feynman: A physics-inspired method for symbolic regression. *Science Advances*, 6(16):eaay2631.

- Uy, N. Q., Hoai, N. X., O’Neill, M., McKay, R. I., and Galván-López, E. (2011). Semantically-based crossover in genetic programming. *Genetic Programming and Evolvable Machines*, 12(2):91–119.
- Wei, J., Wang, X., Schuurmans, D., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35.
- Wolpert, D. H. and Macready, W. G. (1997). No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82.
- Xu, K., Zhang, M., Li, J., et al. (2021). How neural networks extrapolate: From feedforward to graph neural networks. *ICLR 2021*.

Appendix A. Full Benchmark Case Definitions

A.1 Automated Market Makers

- **Constant product formula:** $x \cdot y = k$. Tests basic AMM invariant preservation.
- **Spot price from AMM:** $P = y/x$. Algebraic; should be trivially recovered.
- **Spot price from AMM reserve:** $P = \Delta y / \Delta x$. Marginal price from reserve changes.
- **AMM output amount:** $\Delta y = y - k / (x + \Delta x)$. Tests multi-step algebraic composition.
- **Price slippage percentage:** $\text{slip} = (P_{\text{exec}} - P_{\text{spot}}) / P_{\text{spot}}$.
- **Constant Product Price Impact:** quadratic impact formula.
- **LP share percentage:** $s = \Delta L / L_{\text{total}}$.
- **LP fee earnings:** $e = s \cdot F \cdot V$, where F is fee tier and V volume.
- **Impermanent loss percentage:** $\text{IL} = 2\sqrt{r} / (1 + r) - 1$, where r is the price ratio at withdrawal vs. deposit.
- **Impermanent loss in constant product:** related quantity for the constant-product setting specifically.
- **Impermanent loss breakeven fee rate:** the fee rate F at which LP fees exactly offset impermanent loss.
- **Uniswap V3 virtual:** virtual liquidity quantity for concentrated positions.
- **Concentrated liquidity position width:** tick range formula.
- **Capital efficiency:** ratio of effective liquidity to deposited capital.

A.2 Lending Protocols

- **Loan-to-Value:** $LTV = D/C$.
- **Health factor:** $h = C_{\text{adj}}/D$.
- **Collateral ratio:** $CR = C/D$.
- **Reserve ratio:** $\rho = R/(R + L)$.
- **Utilization rate of DeFi:** $u = L/(L + R)$.
- **Borrow APY from utilization:** kinked interest rate model.
- **Supply APY from borrow:** $r_s = r_b \cdot u$.
- **Partial liquidation amount:** amount of collateral seized.
- **Liquidation Price Long and Liquidation Price Short:** prices at which a position is liquidated.
- **Leveraged position notional:** $N = C \cdot L$, where L is leverage.
- **Effective Leverage:** $L_{\text{eff}} = N/C$.
- **Maximum safe leverage:** function of maintenance margin rate.
- **Required collateral:** collateral needed for a given position.
- **Cross-margin available balance:** net balance accounting for all positions.
- **Multi-Collateral LTV:** weighted LTV across multiple collateral types.
- **Liquidation price for leveraged long and leveraged short:** extended formulas accounting for funding rates.

A.3 Derivatives Pricing

- **Black–Scholes Call Price:** $C = S\Phi(d_1) - Ke^{-rT}\Phi(d_2)$, with $d_{1,2} = [\ln(S/K) + (r \pm \frac{1}{2}\sigma^2)T]/(\sigma\sqrt{T})$.
- **Black–Scholes Put Price:** $P = Ke^{-rT}\Phi(-d_2) - S\Phi(-d_1)$.
- **Delta:** $\Delta = \Phi(d_1)$ (call).
- **Gamma:** $\Gamma = \phi(d_1)/(S\sigma\sqrt{T})$.
- **Theta:** $\Theta = -S\phi(d_1)\sigma/(2\sqrt{T}) - rKe^{-rT}\Phi(d_2)$.
- **Vega:** $\mathcal{V} = S\phi(d_1)\sqrt{T}$.
- **Put-call parity:** $C - P = S - Ke^{-rT}$.
- **Forward price for derivative:** $F = Se^{rT}$.

- **Call option intrinsic:** $\max(S - K, 0)$.
- **Simple options moneyness:** $(S - K)/K$.
- **Convexity Adjustment:** bond convexity formula.

A.4 Risk and Portfolio Metrics

- **Value at Risk at 95 % and 99 %:** parametric VaR formulas.
- **Expected Shortfall at 95 % and 99 %:** conditional VaR formulas.
- **Multi-day Value at Risk:** $\text{VaR}_T = \text{VaR}_1 \cdot \sqrt{T}$.
- **ES scaling for multi-day:** analogous scaling for ES.
- **Incremental VaR:** marginal contribution of one asset to portfolio VaR.
- **Portfolio VaR for two correlated assets:** $\text{VaR}_P = \sqrt{w_1^2 \sigma_1^2 + w_2^2 \sigma_2^2 + 2w_1 w_2 \rho \sigma_1 \sigma_2}$.
- **Correlated Portfolio VaR:** multivariate extension.
- **Component ES:** ES contribution decomposed by asset.
- **Portfolio Expected Shortfall for correlated assets:** the most complex risk case; requires integration over a multivariate Gaussian tail.
- **Annualised Portfolio tracking error:** $\text{TE} = \sigma_\epsilon \sqrt{252}$.
- **Portfolio Sharpe Ratio:** $(r_p - r_f)/\sigma_p$.
- **Information ratio:** $\text{IR} = \alpha/\text{TE}$.
- **Position margin ratio:** margin / position notional.

A.5 Staking and Protocol Mechanisms

- **Simple Staking APY:** $\text{APY} = r$.
- **Compounding Staking Returns:** $(1 + r/n)^{nT} - 1$.
- **Staking reward for fixed lock-up:** $R = P \cdot r \cdot T$.
- **Validator commission adjusted:** post-commission yield.
- **Slashing penalty:** fractional penalty applied to staked balance.
- **Protocol reserve accumulation:** cumulative reserve growth formula.
- **Funding rate cost:** $C = N \cdot f \cdot h$, where f is the funding rate and h is the holding period in funding intervals.
- **APY calculation:** annualised yield from periodic rates.

Table 10: HypatiaX benchmark version history and key changes.

Version	Cases	Key Changes
v1.0	62	Initial benchmark; axis-aligned splits; no trust gating.
v2.0	71	PCA-directed splits introduced; trust gate added ($R^2 > 0.1$).
v3.0	74	Three hard cases added; trust gate raised to $R^2 > 0.5$; data leakage fixed; unified executor.

Appendix B. Benchmark Version History

Appendix C. Corrected Runtime Claim: Detailed Analysis

The following table supports the analysis in the runtime note on p. 1 and the Contributions list (item iii).

Table 11: Detailed timing comparison and speedup calculations (v3.0 benchmark).

Comparison	LLM time (s)	NN time (s)	Hybrid time (s)	Speedup
Mean (all 74 cases)	11.4	3.0	6.8	Hybrid 2.30× <i>slower</i> than NN
Median (all 74 cases)	10.3	2.7	1.7	Hybrid 1.64× faster than NN
LLM-routed only ($n = 68$)	—	2.7	1.56*	Hybrid 1.73× faster than NN

*Estimated from 1.73× speedup vs. NN median of 2.7 s.

Previously claimed: 3.7× speedup = 73% reduction. **Not supported by data.**

The source of the erroneous 73% claim is unclear; it does not correspond to any of the measured speedup values. The most likely explanation is that an earlier benchmark run with a different set of cases (fewer hard cases requiring NN fallback) produced a more favourable mean speedup, and this figure was carried forward without re-verification against v3.0 data.